

FlexLogger Plug-In Development Kit User Manual



FlexLogger Plug-In Development Kit User Manual

Contents

- 1.1 FlexLogger Plug-In Development Kit User Manual 4
- 1.2 Overview 6
 - 1.2.1 Key Features 6
 - 1.2.2 Components of the FlexLogger Plug-In Development Kit 6
- 1.3 Requirements 8
- 1.4 New Features and Changes 10
 - 1.4.1 FlexLogger Plug-In Development Kit 2025 Q3 10
 - 1.4.2 FlexLogger Plug-In Development Kit 2025 Q2 10
 - 1.4.3 FlexLogger Plug-In Development Kit 2024 Q3 10
 - 1.4.4 FlexLogger Plug-In Development Kit 1.7 10
- 1.5 Updates and Changes for FlexLogger Plug-In Development Kit
Extended Support Versions 12
 - 1.5.1 FlexLogger Plug-In Development Kit 1.6 12
 - 1.5.2 FlexLogger Plug-In Development Kit 1.5 12
 - 1.5.3 FlexLogger Plug-In Development Kit 1.4 12
 - 1.5.4 FlexLogger Plug-In Development Kit 1.3 12
 - 1.5.5 FlexLogger Plug-In Development Kit 1.2 12
 - 1.5.6 FlexLogger Plug-In Development Kit 1.1 13
- 1.6 Installing the FlexLogger Plug-In Development Kit 14
- 1.7 Creating a FlexLogger Plug-In from a Template 16
- 1.8 Examples 20
- 1.9 FlexLogger Plug-In File Structure 22
 - 1.9.1 Loading FlexLogger Plug-ins from Custom Locations 23
- 1.10 Using Plug-Ins in FlexLogger 24
 - 1.10.1 Plug-In and Channel Parameters 25
- 1.11 FlexLogger Plug-In Development 28
 - 1.11.1 Accessing the FlexLogger Palette in LabVIEW 30
 - 1.11.2 FlexLogger Plug-In LabVIEW Classes 32
 - 1.11.2.1 Processing Element Class 32
 - 1.11.2.2 Plug-in Class 35
 - 1.11.2.3 Channel Class 36
 - 1.11.2.4 System Interface Class 50

- 1.11.2.5 Additional Classes 50
- 1.12 FlexLogger Plug-In Debugging 52
 - 1.12.1 Debugging Plug-In Loading Errors 52
 - 1.12.2 Viewing Errors in FlexLogger 52
 - 1.12.3 Interactively Testing Plug-Ins 53
 - 1.12.3.1 Interactive Debugging Session Example 59

FlexLogger Plug-In Development Kit User Manual

The FlexLogger Plug-In Development Kit User Manual provides detailed descriptions of the product functionality and the step by step processes for use.

Looking for Something Else?

For information not found in the User Manual for your product, such as specifications and API reference, browse *Related Information*.



RELATED INFORMATION

[*FlexLogger Plug-In Development Kit Compatibility with FlexLogger and LabVIEW*](#)

[*Download the FlexLogger Plug-In Development Kit*](#)

[*License Setup and Activation*](#)

[*FlexLogger Plug-In Development Kit Release Notes*](#)

[*FlexLogger User Manual*](#)

[*FlexLogger Python API Overview*](#)

[*FlexLogger Python API Reference*](#)

[*FlexLogger Plug-In Development Kit API Reference*](#)

FlexLogger Plug-In Development Kit Overview

The FlexLogger Plug-In Development Kit is a FlexLogger add-on that is designed for test engineers. Use the FlexLogger Plug-In Development Kit to create LabVIEW plug-ins that perform custom in-line calculations. The plug-ins can acquire data from NI hardware that is supported by FlexLogger and third-party hardware.

Key Features

- Easy integration with FlexLogger
- Plug-in templates to:
 - communicate with NI hardware supported by FlexLogger and third-party hardware
 - perform custom in-line calculations
 - generate waveform data

Components of the FlexLogger Plug-In Development Kit

The FlexLogger Plug-In Development Kit is an optional add-on designed for use in a FlexLogger test system.

- FlexLogger
- LabVIEW
- LabVIEW Application Builder Module

FlexLogger Plug-In Development Kit Requirements

The FlexLogger Plug-In Development Kit works with LabVIEW and FlexLogger. Refer to the LabVIEW documentation and FlexLogger documentation to ensure your system meets the specified minimum requirements.

Software Version Compatibility

Refer to *FlexLogger Plug-In Development Kit Compatibility with FlexLogger and LabVIEW* for an updated list of compatible software versions.



RELATED INFORMATION

[*FlexLogger Plug-In Development Kit Compatibility with FlexLogger and LabVIEW*](#)

[*FlexLogger System Requirements*](#)

[*LabVIEW System Requirements*](#)

FlexLogger Plug-In Development Kit New Features and Changes

Learn about updates, including new features and behavior changes, introduced in each version of FlexLogger Plug-In Development Kit.

Discover what is new in the latest releases of FlexLogger Plug-In Development Kit.



NOTE

If you cannot find new features and changes for your version, it might not include user-facing updates. However, your version might include non-visible changes such as bug fixes and compatibility updates. For information about non-visible changes, refer to your product *Release Notes*.

FlexLogger Plug-In Development Kit 2025 Q3

- Find detailed API reference documentation for the FlexLogger Plug-In Development Kit in-product or online at ni.com/docs. Refer to the *FlexLogger Plug-In Development Kit API Reference* for more information.



RELATED INFORMATION

[FlexLogger Plug-In Development Kit API Reference](#)

FlexLogger Plug-In Development Kit 2025 Q2

- Added support for LabVIEW 2025

FlexLogger Plug-In Development Kit 2024 Q3

- Added support for LabVIEW 2024

FlexLogger Plug-In Development Kit 1.7

- Added support for LabVIEW 2023 Q1

Updates and Changes for FlexLogger Plug-In Development Kit Extended Support Versions

Browse updates and changes made in FlexLogger Plug-In Development Kit versions on extended support.



NOTE

If you cannot find changes for your version, it might be a more recent version, documented as a new feature. Or, your version might not have included user-facing updates. You can find more information about non-visible changes, such as bug fixes, compatibility updates, and stability adjustments or maintenance adjustments, in the product *Release Notes*, available on ni.com.

FlexLogger Plug-In Development Kit 1.6

- Send strings from third party devices to FlexLogger. Use the Produce Data for FlexLogger plug-in template and configure the Write Data VI to write string channel data.



RELATED INFORMATION

[Writing Channel Data from a Plug-In to FlexLogger](#) on page 37

FlexLogger Plug-In Development Kit 1.5

- Added support for LabVIEW 2022

FlexLogger Plug-In Development Kit 1.4

- Added support for LabVIEW 2021

FlexLogger Plug-In Development Kit 1.3

- Added support for LabVIEW 2020

FlexLogger Plug-In Development Kit 1.2

- Support for digital input plug-in channels

FlexLogger Plug-In Development Kit 1.1

- New Perform calculation plug-in template type
 - Send FlexLogger analog input channels
 - Publish calculation results back to FlexLogger with the same timing information as the mapped channels
- Custom plug-in commands offer users a new way to interact with plug-ins

Installing the FlexLogger Plug-In Development Kit

Install the FlexLogger Plug-In Development Kit using NI Package Manager.

Before you begin

Before you begin, refer to *FlexLogger Plug-In Development Kit Compatibility with FlexLogger and LabVIEW* to verify software compatibility.

About this task

Use NI Package Manager to download and install your NI software. For more information, refer to *Installing, Updating, Repairing, and Removing NI Software*.

Procedure

1. Install LabVIEW.
2. Install FlexLogger.
3. Install the FlexLogger Plug-In Development Kit.



RELATED INFORMATION

[*Installing, Updating, Repairing, and Removing NI Software with Package Manager*](#)
[*FlexLogger Plug-In Development Kit Compatibility with FlexLogger and LabVIEW*](#)

Creating a FlexLogger Plug-In from a Template

About this task

Use the FlexLogger Plug-in Wizard and plug-in templates to create plug-ins that you can use in FlexLogger.

Procedure

1. Launch LabVIEW.
2. In LabVIEW, select **File»Create Project**.
3. From the **Create Project** window, select **FlexLogger IO Plug-in** and click **Finish**.
4. In the **Create FlexLogger IO Plug-in** window, update the **IO Plug-in Name** and select a **Plug-in Template** based on your goal.

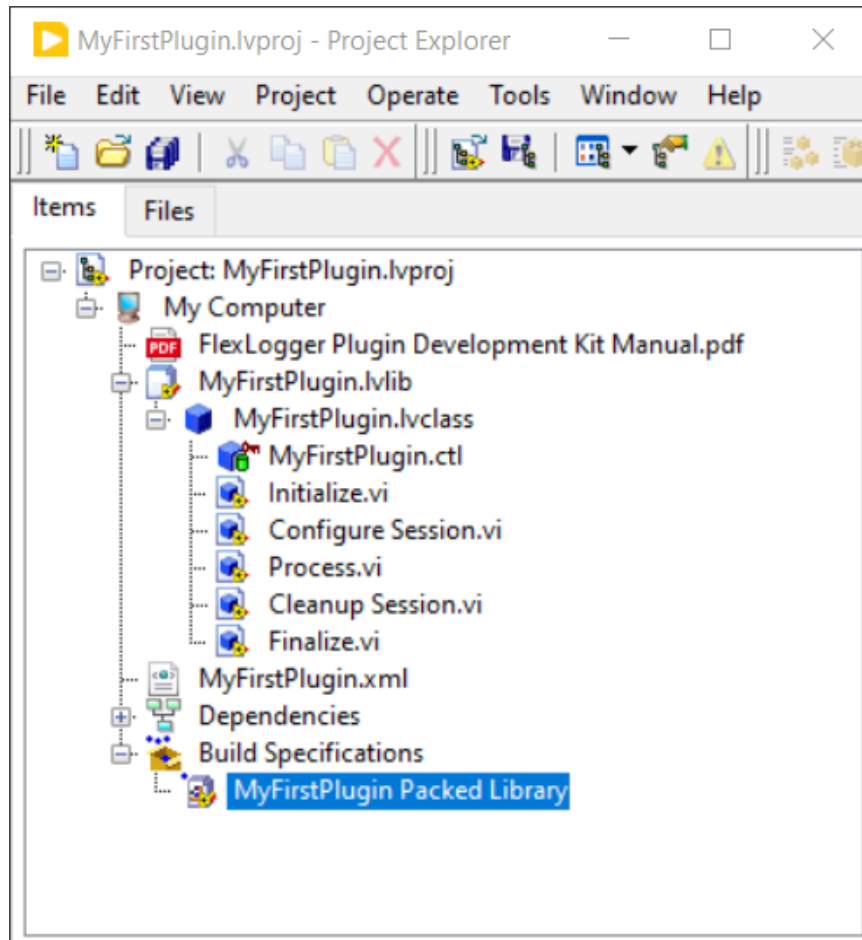
Table 1 : FlexLogger Plug-In Templates

Goal	Plug-In Template	Data Flow
Create a plug-in that sends a setpoint value you enter in FlexLogger to a plug-in channel. Control a third-party instrument with FlexLogger. For more information, refer to <i>Read Setpoint Channels from FlexLogger</i> .	Consume setpoint data	User input → plug-in
Create a plug-in that consumes one or more waveforms from FlexLogger channels, performs a custom calculation, and produces a waveform of resulting data that can be logged in FlexLogger.	Perform calculation	FlexLogger → plug-in and plug-in → FlexLogger
Create a plug-in that generates waveform data and sends it to FlexLogger.	Produce data for FlexLogger	plug-in → FlexLogger

5. Optional: Update the **Description** and **Destination Directory (Source)** for the plug-in. By default, the wizard saves new plug-ins in the `My Documents\LabVIEW Projects\FlexLogger IO Plug-ins\` directory.
6. Click **Create**.

The FlexLogger Plug-In Wizard creates a LabVIEW project with necessary dependencies, a new LabVIEW class, an XML file, and a packed library build specification.

7. Optional: Customize your plug-in using the methods described in the *Plug-in Development* section.
8. To build your plug-in, select the Packed Library build specification and click **Build Selected**.



FlexLogger Plug-in Wizard builds the packed library file and XML file for the plug-in and saves them in the default FlexLogger plug-in directory, %public%\Documents\National Instruments\FlexLogger\Plugins\IOPlugins.

What to do next

After creating a plug-in, use it in your FlexLogger project.



RELATED INFORMATION

[*Read Setpoint Channels from FlexLogger*](#) on page 40

[*FlexLogger Plug-In Development*](#) on page 28

Learn about the components of a plug-in so you can customize a plug-in you created from a template or develop a new plug-in.

[*Using Plug-Ins in FlexLogger*](#) on page 24

Add and configure plug-in channels to your FlexLogger project so you can perform custom in-line calculations or collect data from third-party hardware devices.

FlexLogger Plug-In Development Kit Examples

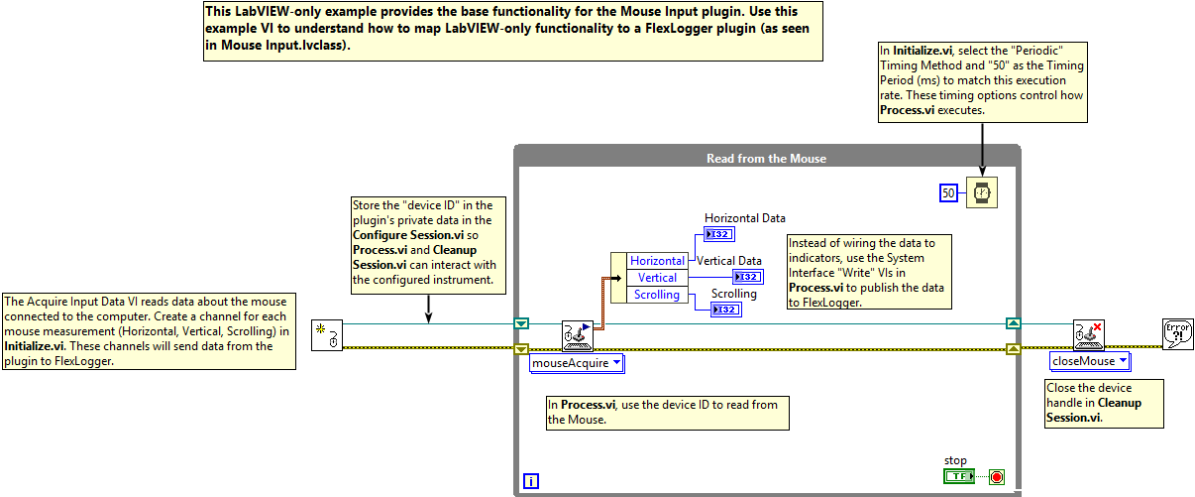
Learn about the various examples included in the FlexLogger Plug-In Development Kit. The examples are located in the %ProgramFiles%\National Instruments\LabVIEW <version>\examples\FlexLogger\IO Plugins directory.

Table 2 : FlexLogger Plug-In Development Kit Examples

Example	Type	Description
Cycle Counter	Perform calculation	Counts the number of times a specified channel rises above a threshold.
Digital Port Calculator	Perform calculation	Combines eight individual digital channels into a single analog value representing the port value.
IVI DC Power Supply	Consume setpoint data	Uses the IVI DC Power class driver to control a power supply instrument. To load this IVI example, first install the IVI Compliance Package from NI Package Manager and an IVI DC Power-compliant instrument driver to control the power supply.
IVI Meter – DC Voltage	Produce data for FlexLogger	Reads DC voltage values from an IVI DMM class driver-compliant instrument. To load this IVI example, first install the IVI Compliance Package from NI Package Manager and an IVI compliant instrument driver to read voltage values from an instrument.
Mouse Input	Produce data for FlexLogger	Reads data from the mouse connected to the computer.
Rosette	Perform calculation	Calculates a rosette measurement from multiple strain input channels.

LabVIEW-only VIs

Some examples include a LabVIEW-only VI that demonstrates how to map functionality from a standalone VI to a FlexLogger plug-in class. For example, the LabVIEW project for the Mouse Input example contains `Mouse Input - LabVIEW Only Example VI` that describes how different parts of the block diagram correspond to the VIs in `Mouse Input.lvclass`.



FlexLogger Plug-In File Structure

Learn where FlexLogger looks for plug-ins so you can load the plug-ins that you created.

The FlexLogger Plug-In Wizard saves the packed library file and the XML file for the plug-in that you built. You can find the files in the default FlexLogger plug-in directory, %public%\Documents\National Instruments\FlexLogger\Plugins\IOPlugins. FlexLogger loads plug-ins from the default directory.

You can configure FlexLogger to load plug-ins from custom locations. Refer to *Loading FlexLogger Plug-ins from Custom Locations* for more information.

A plug-in named MyFirstPlugin, for example, has the following file names and locations.

Table 3 : FlexLogger Plug-In File Locations

Folder	%public%\Documents\National Instruments\FlexLogger\Plugins\IOPlugins\MyFirstPlugin
Packed library file	%public%\Documents\National Instruments\FlexLogger\Plugins\IOPlugins\MyFirstPlugin\MyFirstPlugin.lvlibp
XML file	%public%\Documents\National Instruments\FlexLogger\Plugins\IOPlugins\MyFirstPlugin\MyFirstPlugin.xml FlexLogger uses the XML file to register and to load the plug-in for use at run-time. The XML file contains the plug-in name, description, and version. If you update the plug-in or change the plug-in name, you must manually update the XML file.



NOTE

- The folder, packed library file, and the XML file must have the same name.
- FlexLogger does not support multiple plug-ins with the same name.
- When you modify a plug-in, you must close and re-open your FlexLogger project to use the updated plug-in.

Versioning

When you create a plug-in, the FlexLogger Plug-In Wizard assigns a version number to the plug-in. The wizard defines the version number in the <Version> and <OldestCompatibleVersion> tags in the plug-in XML file. Update the <Version>

and `<OldestCompatibleVersion>` values to track updates to a plug-in that you have already added to a FlexLogger project.



RELATED INFORMATION

[Loading FlexLogger Plug-ins from Custom Locations](#) on page 23

Configure FlexLogger to load plug-ins from locations outside of the default directory.

Loading FlexLogger Plug-ins from Custom Locations

Configure FlexLogger to load plug-ins from locations outside of the default directory.

About this task

By default, FlexLogger loads plug-ins from `%public%\Documents\National Instruments\FlexLogger\Plugins\IOPlugins`. Complete the following steps to load plug-ins from additional, custom locations.

Procedure

1. Using a text editor, create a file that specifies additional plug-in locations using the following JSON format.

```
{ "AdditionalPluginPaths": [ "C:\\Users\\<username>\\Documents\\Flex Logger Projects\\", "<another absolute path>" ] }
```

2. Save the file in the root directory of `%public%\Documents\National Instruments\FlexLogger\Plugins\IOPlugins`. FlexLogger does not have specific requirements for the file name.
3. Change the file extension from `.txt` to `.config`.
4. Re-launch FlexLogger.

Results

The plug-ins you saved in the custom file locations are visible when you select **Add channels»Plug-in** in the **Channel Specification** toolbar.

Using Plug-Ins in FlexLogger

Add and configure plug-in channels to your FlexLogger project so you can perform custom in-line calculations or collect data from third-party hardware devices.

Before you begin

If you do not have a FlexLogger project already, create a FlexLogger project. For more information, refer to *Creating a New Project*.

Procedure

1. Launch FlexLogger and open your project.
2. Complete the following tasks for your desired goal.

Goal	Tasks
Add a plug-in to my project	1. In the Channel Specification toolbar, select Add channels»Plug-in. 2. Select the plug-in you want to add.  NOTE If FlexLogger did not load any plug-ins on launch, the Plug-in option does not appear. For troubleshooting plug-in issues, refer to <i>FlexLogger Plug-In File Structure</i> and <i>FlexLogger Plug-In Debugging</i>. FlexLogger adds the plug-in you selected to the Channel Specification under the local system.
Configure plug-in parameters	Click Configure  at the right edge of the plug-in header to open the plug-in configuration dialog box.  NOTE When FlexLogger is running a test, you cannot modify plug-in parameters, and FlexLogger disables the plug-in Configure gear . FlexLogger automatically generates the plug-in configuration dialog box with settings for the parameters you defined in the plug-in.

Goal	Tasks
Configure channel parameters	1. Hover over an individual channel to see Configure . 2. Click the gear to open the channel configuration dialog box.  NOTE Channel-level parameters apply to only the selected channel. FlexLogger automatically generates the channel configuration dialog box with settings for parameters you defined in the plug-in.
Stop logging a channel	Hover over the channel and click Disable logging ( .
Delete a plug-in	Click Delete  at the right edge of the plug-in header to remove a plug-in from your project.

**RELATED INFORMATION**

[Creating a New Project — FlexLogger User Manual](#)

[FlexLogger Plug-In File Structure](#) on page 22

Learn where FlexLogger looks for plug-ins so you can load the plug-ins that you created.

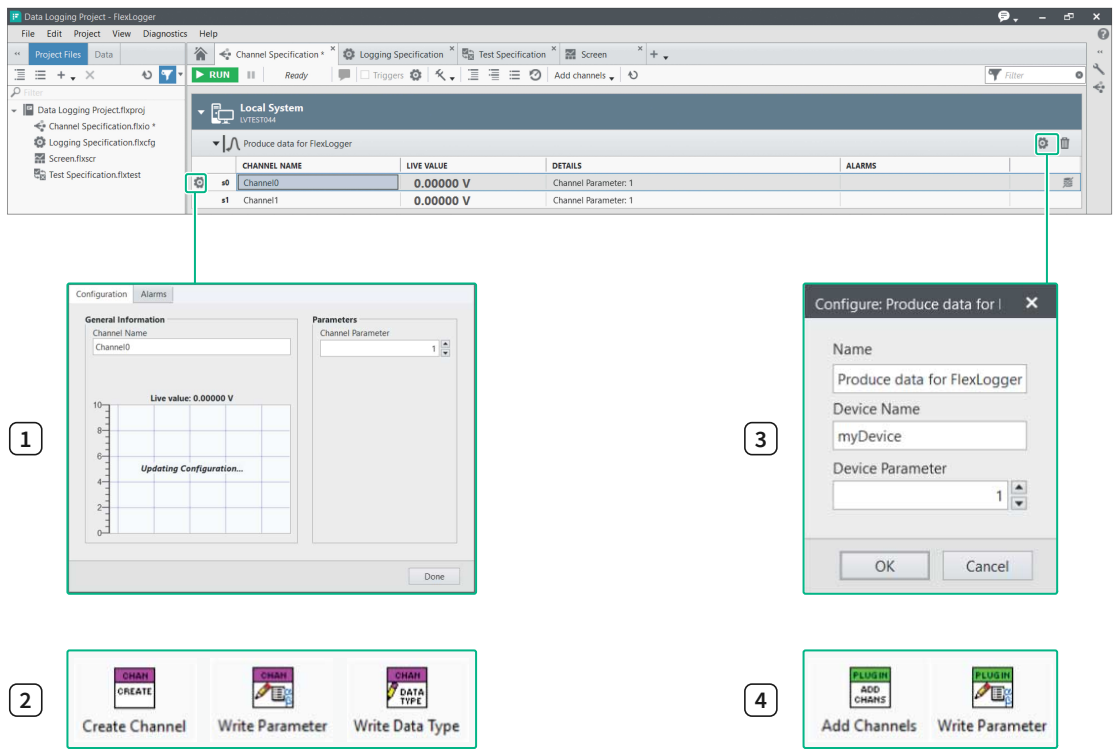
[FlexLogger Plug-In Debugging](#) on page 52

Troubleshoot plug-in problems with the debugging resources available in LabVIEW and FlexLogger.

Plug-In and Channel Parameters

Configure the plug-in parameters and the channel parameters of a plug-in to support custom calculations and data acquisition from third-party hardware devices.

Plug-in parameters, such as **Device Name**, apply to the whole plug-in. *Channel parameters* apply to a specific plug-in channel. The following figure illustrates where you can configure plug-in parameters and channel parameters in FlexLogger and LabVIEW.



1.

FlexLogger dialog box for configuring channel parameters.
2.

LabVIEW VIs for customizing channel parameters.
3.

FlexLogger dialog box for configuring plug-in parameters.
4.

LabVIEW VIs for customizing plug-in parameters.



RELATED INFORMATION

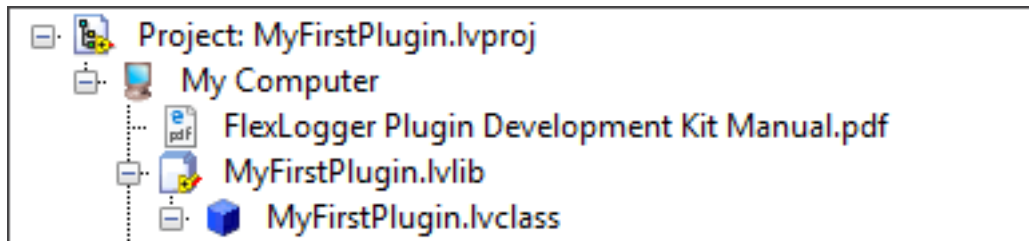
[Accessing the FlexLogger Palette in LabVIEW](#) on page 30

Use the FlexLogger palette in LabVIEW to develop and customize plug-ins.

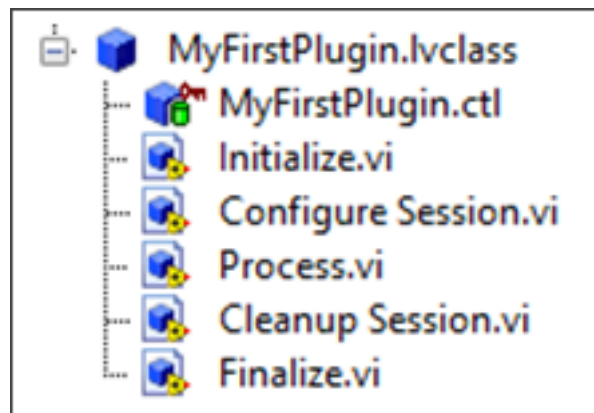
FlexLogger Plug-In Development

Learn about the components of a plug-in so you can customize a plug-in you created from a template or develop a new plug-in.

When you create a plug-in named `MyFirstPlugin`, for example, using the FlexLogger Plug-In Wizard in LabVIEW, the wizard creates a project with the hierarchy shown below.



The VIs within `MyFirstPlugin.lvclass` are the starting point for code changes. These VIs perform different roles, such as defining parameters and exchanging data with FlexLogger. For details about how these VIs function, refer to *Processing Element Class*. State information—like a hardware reference—passes between the VIs using the private class data defined in `MyFirstPlugin.ct1`.



If you plan to develop a plug-in, first test your hardware and drivers using a LabVIEW VI that communicates with third-party instruments. NI recommends that you start with the LabVIEW-only VI provided in some of the examples, including Mouse Input. This LabVIEW-only VI demonstrates how to map functionality from a standalone VI to a FlexLogger plug-in class. For more information, refer to *FlexLogger Plug-In Development Kit Examples*.

Review the examples installed with the FlexLogger Plug-In Development Kit and refer to the table below for guidance on your plug-in related question.

Table 4 : Plug-In Development Guidance

Question	Where to Go
How many channels do I need?	Refer to <i>Channel Class</i> for details on how plug-in channels communicate with FlexLogger.
How many parameters will I need to edit while logging data?	Only setpoint channels can be edited while data logging is in progress. Refer to <i>Read Setpoint Channels from FlexLogger</i> for details on how to use setpoint channels.
How many plug-in parameters do I need?	Plug-in parameter values affect the entire plug-in—for example, the name of a hardware resource. Refer to <i>Plug-in Class</i> for more information.
How many channel-specific parameters do I need?	Channel-specific parameter values are unique to every channel and are defined when creating new plug-in channels. Refer to <i>Channel Class</i> for more information.
What data do I need to pass between plug-in VIs?	You must pass references to any resources you initialize or allocate. For example, if an instrument session is initialized in <i>Configure Session VI</i> , it needs to be passed to <i>Process VI</i> for use and then to <i>Cleanup Session VI</i> for disposal. Important plug-in information can be stored in the private class data of the plug-in (for example, <code>MyFirstPlugin.ct1</code>).
How do I know my plug-in works?	Use one of the following approaches to evaluate the functionality of your plug-in by executing the logic and investigating the effect of code changes. • Load the plug-in into FlexLogger. Refer to <i>Using Plug-Ins in FlexLogger</i> for more information. • Run the plug-in with the Plug-in Environment Simulator. Refer to <i>Interactively Testing Plug-Ins</i> for more information.



RELATED INFORMATION

[Processing Element Class](#) on page 32

Learn how execution states are managed by FlexLogger plug-ins.

[FlexLogger Plug-In Development Kit Examples](#) on page 20

Learn about the various examples included in the FlexLogger Plug-In Development Kit. The examples are located in the %ProgramFiles%\National Instruments\LabVIEW <version>\examples\FlexLogger\IO Plugins directory.

[Channel Class](#) on page 36

[Read Setpoint Channels from FlexLogger](#) on page 40

[Plug-in Class](#) on page 35

[Using Plug-Ins in FlexLogger](#) on page 24

Add and configure plug-in channels to your FlexLogger project so you can perform custom in-line calculations or collect data from third-party hardware devices.

[Interactively Testing Plug-Ins](#) on page 53

Accessing the FlexLogger Palette in LabVIEW

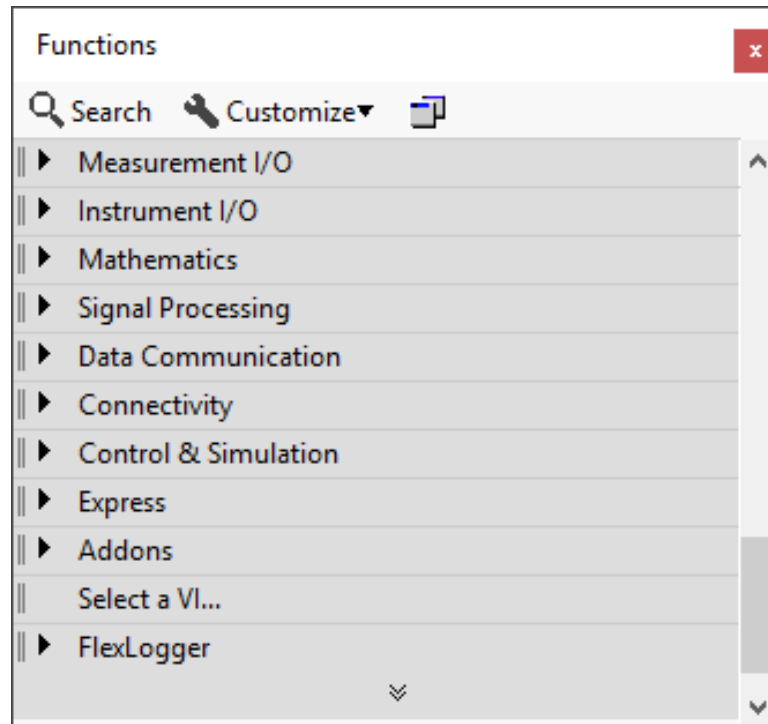
Use the FlexLogger palette in LabVIEW to develop and customize plug-ins.

About this task

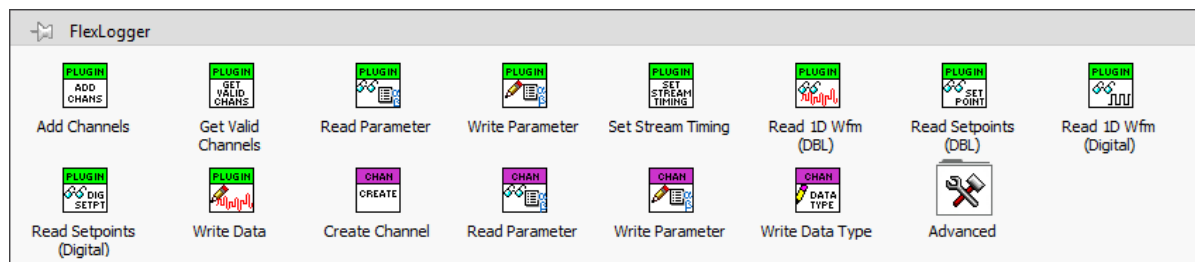
The FlexLogger Plug-In Development Kit installer adds the FlexLogger palette to LabVIEW. The palette provides the plug-in and channel methods needed to interact with FlexLogger.

Procedure

1. In LabVIEW, navigate to **View»Functions Palette**.



2. In the **Functions** window, select the **FlexLogger** palette.



What to do next

For more information on the methods contained in the palette, refer to *FlexLogger Plug-In LabVIEW Classes*.



RELATED INFORMATION

[FlexLogger Plug-In LabVIEW Classes](#) on page 32

In the FlexLogger Plug-In Development Kit, `PluginSDK.lvlib` contains the classes and VIs to use for plug-in development. The most common classes for plug-in development are the `Plug-in` and `Channel` classes.

FlexLogger Plug-In LabVIEW Classes

In the FlexLogger Plug-In Development Kit, `PluginSDK.lvlib` contains the classes and VIs to use for plug-in development. The most common classes for plug-in development are the `Plug-in` and `Channel` classes.



TIP
If you are unfamiliar with LabVIEW object-oriented programming, refer to *LabVIEW Object-Oriented Programming* in the *LabVIEW User Manual* for more information.



RELATED INFORMATION
[LabVIEW Object-Oriented Programming](#)

Processing Element Class

Learn how execution states are managed by FlexLogger plug-ins.

The Processing Element class, `ProcessingElement.lvclass`, defines the plug-in execution states. The FlexLogger data engine manages the transitions between these states with a set of methods. Refer to the following figure and table to learn more about each method.

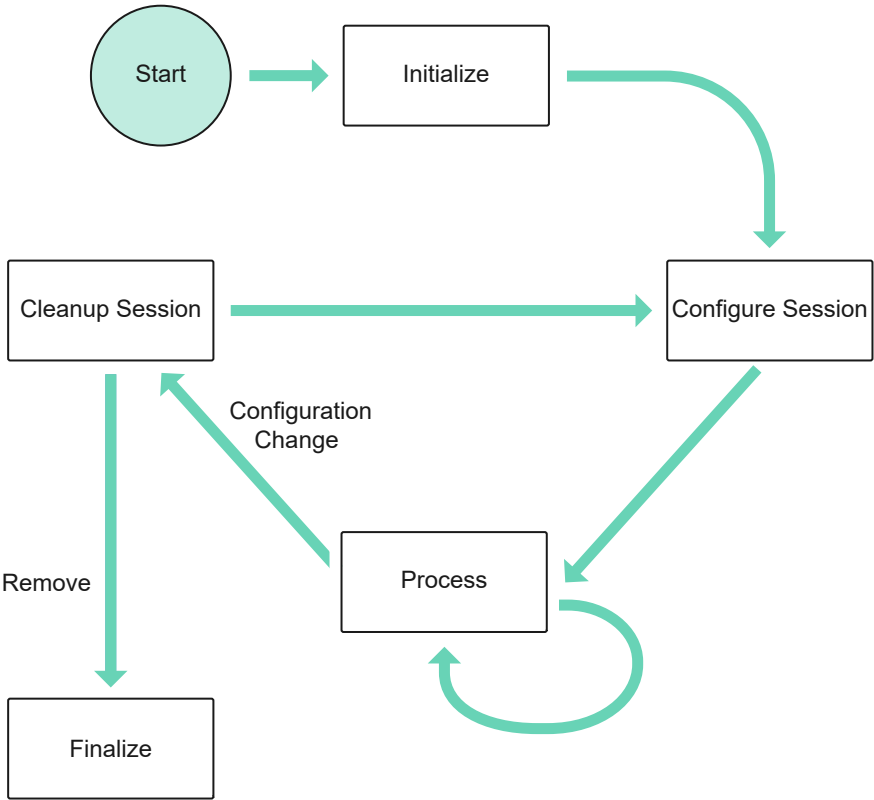


Table 5 : Processing Element Class Methods

Method	Description
Initialize	Runs only once—When you add a plug-in to the FlexLogger Channel Specification.
	Defines the timing method used by the <i>Process</i> method.
	Declare the plug-in parameters users interact with in FlexLogger.
	Declare the channels (and corresponding channel parameters) the plug-in will read data from or send data to.
Configure Session	Runs when you make a configuration change while FlexLogger is not running a test.
	Initialize and configure hardware/instrument sessions. Runs after the <i>Initialize</i> method.
	Read the latest channel parameters set in FlexLogger. Read the latest plug-in parameters set in FlexLogger.
	Set the timing information (dt, T0) for data written to FlexLogger in the <i>Process</i> method.
Process	Read what channels FlexLogger reports as valid (channels defined by the plug-in may be modified in FlexLogger).
	Read from and write to third-party hardware.
	Read channel data from FlexLogger.
Cleanup Session	Write channel data to FlexLogger.
	Runs when you make a configuration change.
Finalize	Cleans up the hardware sessions and resources initialized by the <i>Configure Session</i> method.
	Runs when you remove a plug-in from your FlexLogger project or close the project.
	Cleans up any resources initialized during the <i>Initialize</i> method. Commonly empty.

Process Method Timing

In the *Initialize* method, plug-ins which produce data or consume only setpoint data set the **Timing Parameters** that control how the *Process* method executes. If a plug-in reads waveform data from other FlexLogger channels, you should instead set the **Resampling Parameters**, which determine how input data will be resampled. Plug-ins should typically only use either the Set Plugin Timing VI or the Set Plugin Resampling VI in *Initialize* VI, but not both. In situations where the plug-in timing is not driven by

the timing of the input data, you may want to use the **Set Plugin Timing VI** in addition to **Set Plug-in Resampling VI**.

Table 6 : FlexLogger Plug-in Process Method Timing Parameters

Timing Parameter	Timing Period (ms) required?	Description
Periodic	Yes	With a timing period of <i>n</i> , periodic timing ensures at least <i>n</i> milliseconds between runs of the Process method.
Immediate	No	Run Process without delay. This method is typically used when a blocking call inside the Process method controls timing.
On Data Ready	Optional	For use by plug-ins that read channel data from FlexLogger—such as the Consume setpoint data from FlexLogger template. On Data Ready blocks execution of Process until FlexLogger sends new channel data to the plug-in. This parameter uses a timing period of <i>n</i> milliseconds as a <i>time-out</i> . After passing the time-out period, the plug-in executes Process without receiving new channel data from FlexLogger. If the time-out period is ≤ 0 ms, On Data Ready blocks execution of Process until FlexLogger sends new channel data to the plug-in.
Triggered	No	Not applicable for IO Plug-in development.

Table 7 : FlexLogger Plug-in Process Method Resampling Parameters

Resampling Parameter	Description
Resampling Mode	Determines how resampling of input waveforms will be done. In Minimum and Maximum mode the resampling rate is determined by the input channels. In Maximum mode any slower channels will be resampled to the rate of the fastest channel. In Custom mode resampling is done at a fixed rate specified by the Resampling Rate parameter. In Original Rate mode no resampling will be done. This mode should only be used if there is a single input channel.
Resampling Rate (Hz)	The rate to resample the data at if using Custom mode.
Drift Tolerance (seconds)	The amount of history to store in the plug-in waiting for time aligned data from different sources. A high number here increases the memory usage of the plug-in, a low number potentially causes issues with plug-ins which are not synchronized.
Block Size (seconds)	The minimum amount of overlapping data that should be present before the Process method is called.

Plug-in Class

The Plug-in class, `Plugin.lvclass`, is the parent class for plug-ins created with the FlexLogger Plug-in Development Kit. It provides the state management functionality for the plug-in by inheriting from `Processing Element.lvclass`. The Plug-in class also provides methods to define and interact with plug-in channels and parameters. After initializing the channels, the Plug-in class maintains channel information that correctly reflects the state of the plug-in channels in FlexLogger.

The Plug-in class supports double-precision floating point numbers, strings, enums, Booleans, and integers as plug-in parameters. Plug-in parameters can be created using the **Write Parameter VI**, as shown in the following example:



NOTE

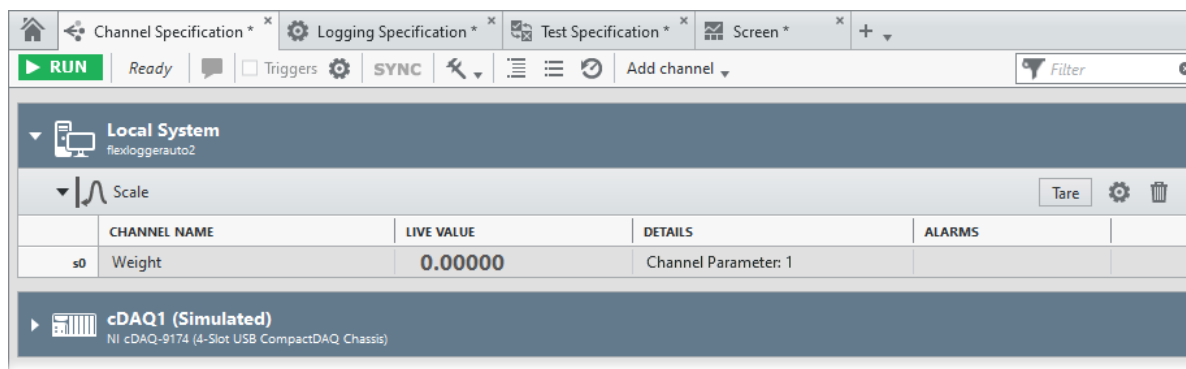
Do not call **Add Channels** multiple times from the same plug-in method as it causes parameter settings to not save correctly.

Commands

Plug-ins have a special parameter type called **Command**. When you select this parameter, the plug-in will create a button in FlexLogger. Pressing the button in FlexLogger sends a notification to the plug-in. For example, use a Command parameter to create a **Tare** button for a scale plug-in.



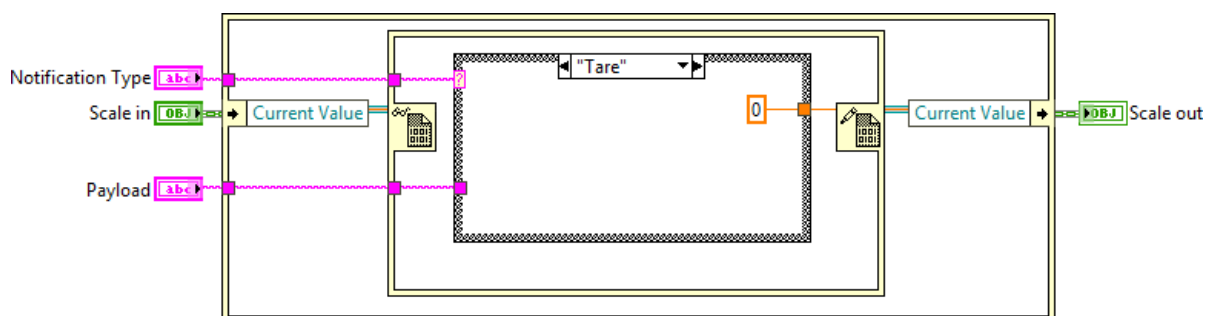
When you press the **Tare** button in FlexLogger, the plug-in tares the scale.



In order to create a user-defined command in a plug-in, you must override the `Handle Notification VI`. Create an overriding VI by right-clicking the `.lvclass` file in the project and selecting **New»VI for Override**.

This VI has two input parameters:

- **Notification Type**- a string input to match the name given to your plug-in command parameter
- **Payload**- a string input that is empty when the notification comes from a command button, except for the built-in commands, Test Start and Test Stop. The Test Start and Test Stop commands return the time (in seconds) for the start or stop of a test. Use `Notification Payload to Timestamp VI` to decode the payload.



To create a special case for Test Start or Test Stop notifications, add these notification type names to the case structure and use `Notification Payload to Timestamp VI` (`<vi.lib>\FlexLogger\SDK\PluginSDK.lvlib\PEFClasses\SDK\Plugin\`) to convert the payload to a timestamp.

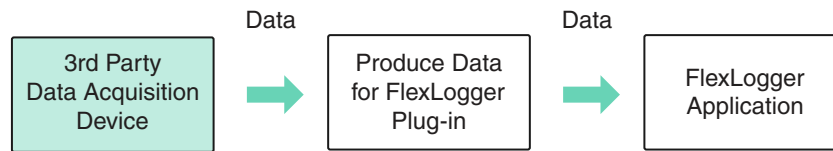
Note that `Handle Notification VI` runs in a separate loop. If you want to share data with other VIs in the Plug-in class, use a Data Value Reference instead of directly changing the private class member variables.

Channel Class

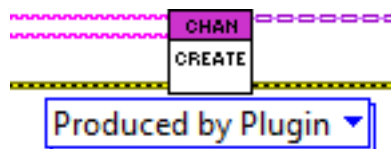
The Channel class, `Channel.lvclass`, allows plug-ins to send data to FlexLogger and read setpoint data from FlexLogger. The Plug-in class uses the Channel class to organize channels and ensure consistency between the plug-in and FlexLogger.

Writing Channel Data from a Plug-In to FlexLogger

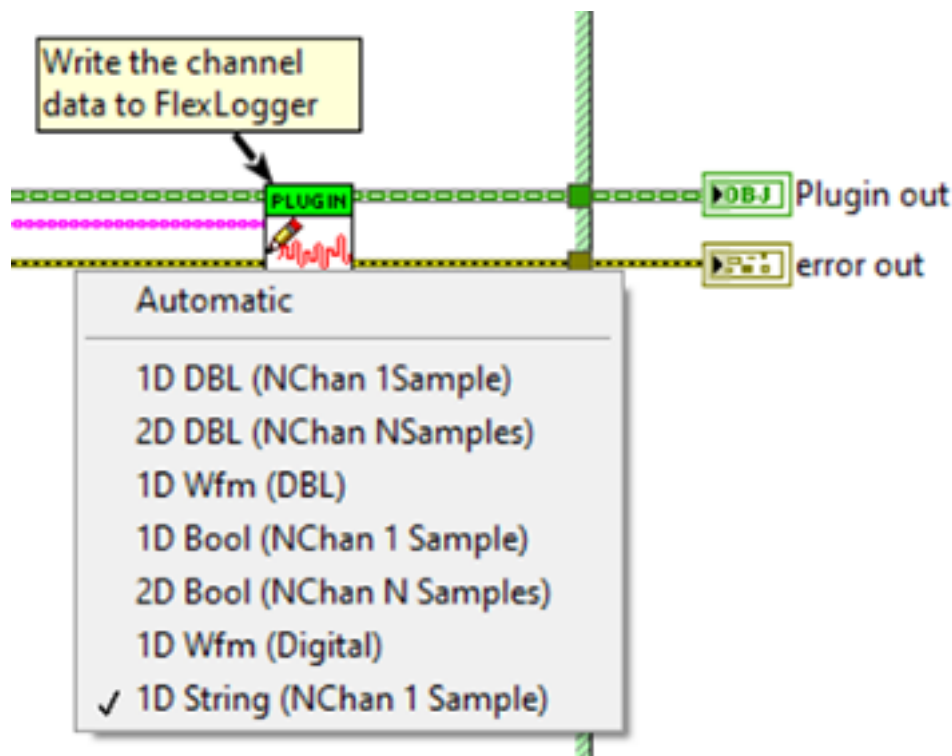
Plug-ins created from the Produce data for FlexLogger template send collected data to FlexLogger. Use the Produce data for FlexLogger template as a starting point for creating your own plug-ins that send channel data to FlexLogger.



To create a channel that sends data to FlexLogger, select the **Produced by Plugin** mode in **Create Channel VI**. The **Create Channel VI** is in the **Initialize** method of the **Produce data for FlexLogger** template.

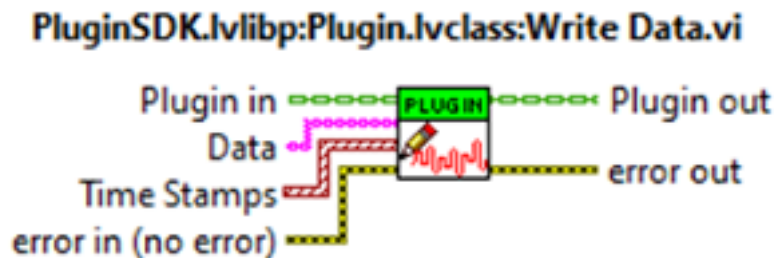


You can write string data or waveform data to a FlexLogger channel. To write string data, specify **1D String (NChan 1 Sample)** as the data type in the **Write Data VI**. The **Write Data VI** is in the **Process** method of the **Produce data for FlexLogger** template.

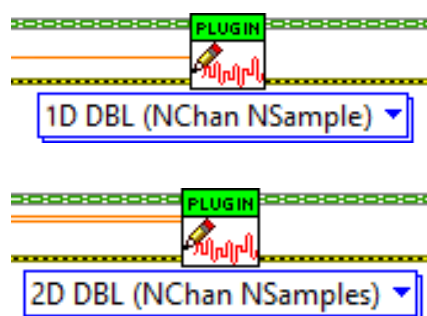


When writing strings, you can optionally specify the time stamp at which the string is written. If you don't specify a time stamp, the VI will use the current time as the time stamp

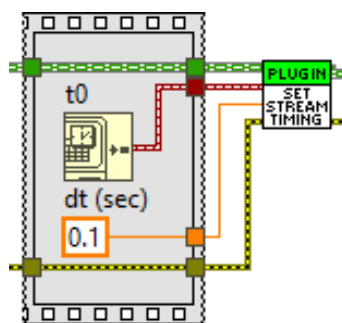
for when the string is written. The following image shows all of the inputs and outputs for the Write Data VI.



To send waveform data, specify either a one or two-dimensional array of doubles.



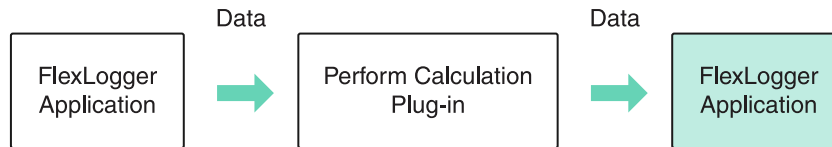
Writing waveforms that overlap in time disrupts visualization and logging capabilities of FlexLogger. The Produce data for FlexLogger template uses the Set Stream Timing VI to send data to FlexLogger without overlap errors. The Set Stream Timing VI is in the Configure Session method and defines the waveform **dt** and **Start Time** timing parameters.



Since the Configure Session method always executes before the Process method, the timing information for the output channel updates after every configuration change.

Read and Write Waveform Channels

Plug-ins created from the Perform calculation template read waveform data from FlexLogger and publish waveform or string data to FlexLogger. You can also create a plug-in which only reads waveform data and does not publish any data back to FlexLogger.



To map FlexLogger channels to a plug-in use the Plug-in class `Write Parameter VI` **Single Channel** or **Multiple Channel** modes. The **Single Channel** mode maps one named parameter to one channel while the **Multiple Channel** mode maps one named parameter to multiple FlexLogger channels.



Select the channels you want to map from the plug-in level **Configure** gear located to the right of the plug-in name.



NOTE

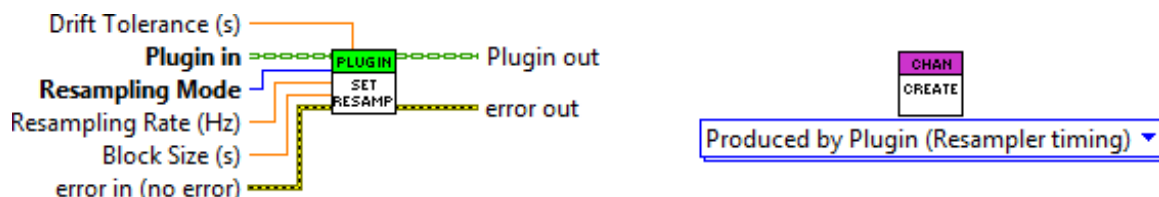
Setpoint channels do not work with waveform parameters.

You can also add support for producer channels to allow users to map other FlexLogger channels to the plug-in by using the Channel class `Write Parameter VI`.



Select the channels you want to map from the channel-in level **Configure** gear that appears to the left of the channel name when you hover over the channel.

When using the minimum or maximum resampling mode of a plug-in, it is common that the rate of the waveform channels produced by the plug-in match the rate of the input. Since the rate can change dynamically based on your channel selection in these modes, a new type of channel, **Produced by Plugin (Resampler Timing)**, is provided which automatically adapts to these changes in sample rate. NI recommends creating your producer channels with this option whenever the resampling mode is set to **Minimum** or **Maximum**.



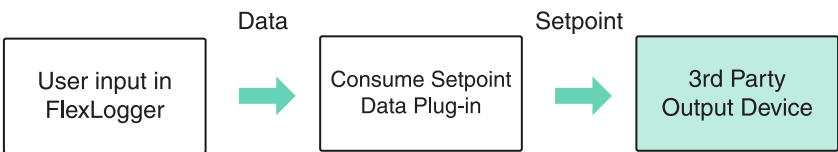


NOTE

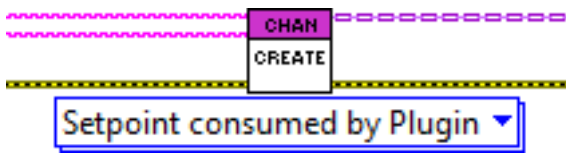
When reading waveforms from other channels, the length of the waveform will be determined by how much data that channel wrote in one cycle of its Process VI. Since the waveforms are not combined, waiting for longer periods between reads does not result in more data in the waveform, but in more waveforms in the queue that can be read by Read 1D Wfm VI.

Read Setpoint Channels from FlexLogger

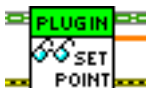
Plug-ins created from the Consume setpoint data template read values from user settable controls in FlexLogger and use those values to control third-party output devices.



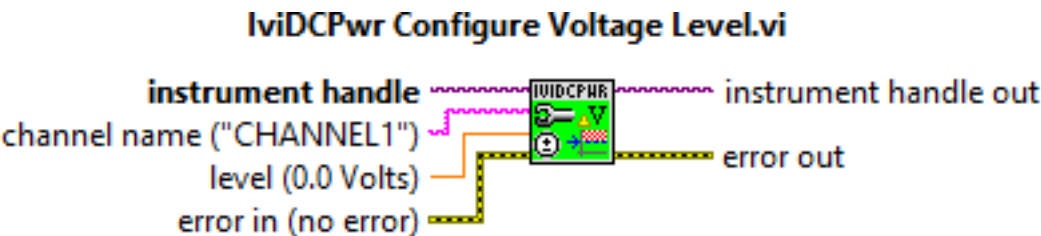
To create a channel that consumes setpoint channel information from FlexLogger, use the **Setpoint consumed by Plugin** option in Create Channel VI.



As demonstrated by the Process VI in the Consume setpoint data template, the Plug-in class Read Latest Setpoints VI extracts the most recent scalar value and provides a one-dimensional array of setpoints (one per channel).



In the IVI DC Power Supply example, the setpoint value read from FlexLogger is used as the **level** input to set the Power Supply DC voltage level.



This VI configures the DC voltage level that the power supply attempts to generate.

Retrieve Data from Individual Channels

To read waveforms from other plug-ins or use a setpoint channel in the Process VI, use either the Read 1D Wfm VI or Read Latest Setpoints VI based on the types of channels that are configured. Use Read 1D Wfm VI to return an array of data for each waveform or setpoint channel. Use Read Latest Setpoints VI to return the latest value for each waveform and setpoint channel.



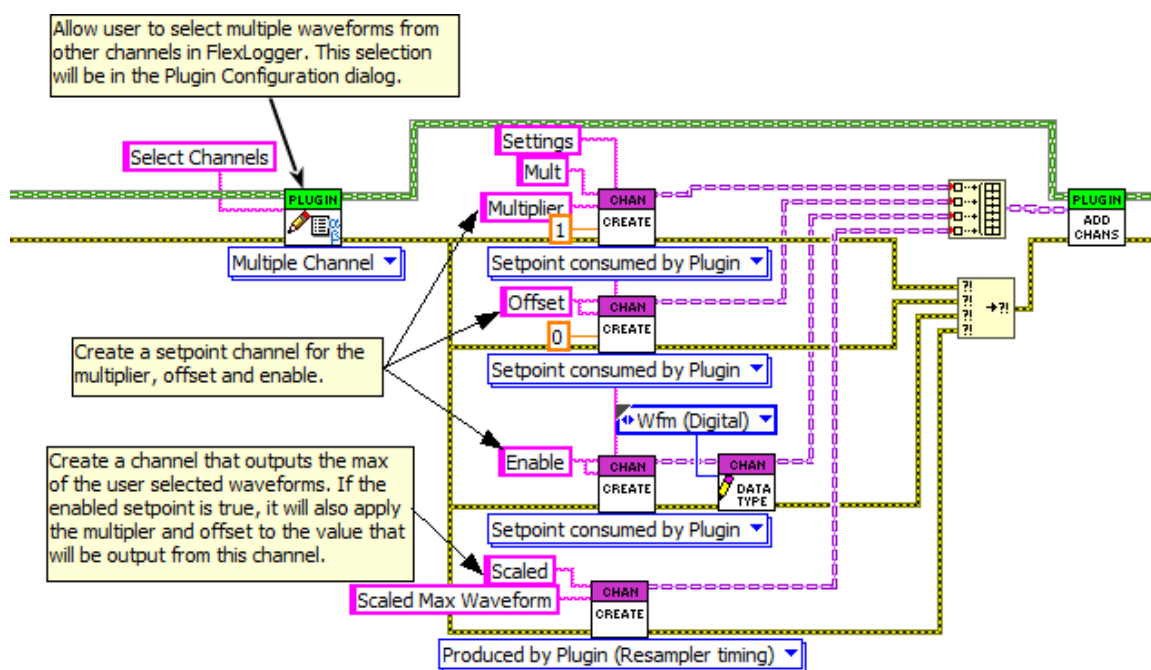
NOTE

You should only use one instance of either of these VIs in the Process VI.

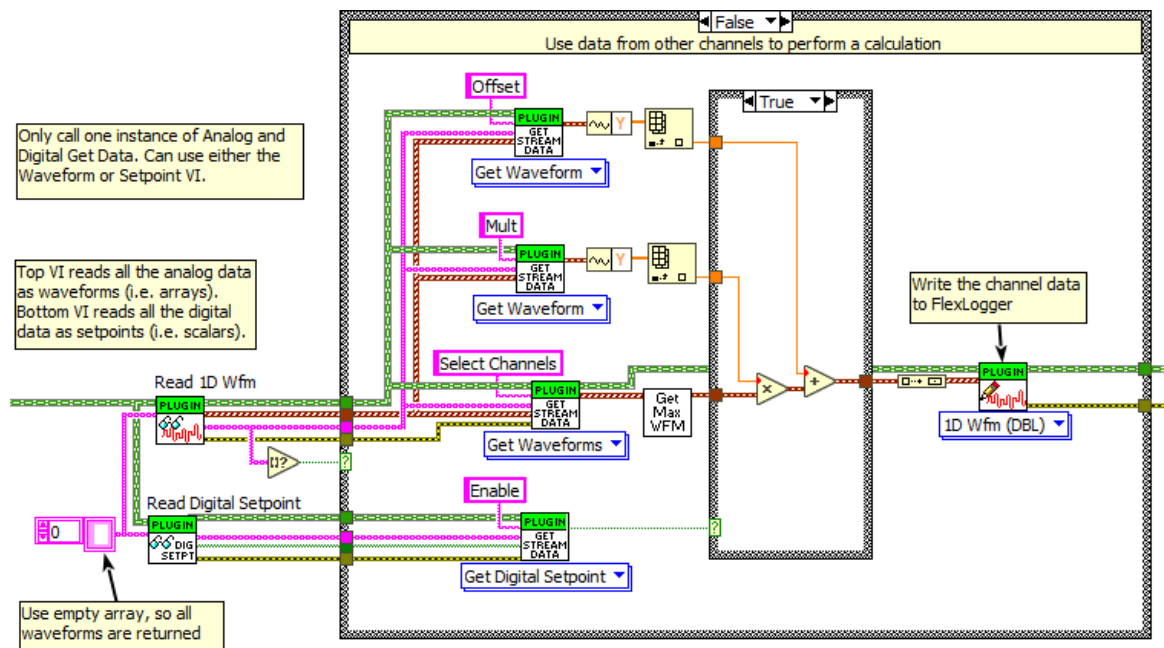
If you have a mix of analog and digital data to read, you can use one of these VIs to read the analog data and one of the corresponding digital VIs to read the digital data.

The following is an example plug-in that scales waveforms from other channels based on setpoint channels. These setpoint channels provide a multiplier, offset, and a digital setpoint to enable or disable the scaling.

The Initialize VI could look like the following to enable the user to select waveforms from other channels in FlexLogger and use setpoint channels to change parameters while logging data.



Based on the configuration of the previous Initialize VI, the Process VI might look like the following:



Read 1D Wfm VI gets all of the analog data and returns it as an array. The **Offset** and **Mult** parameters receive the setpoint channel data of the plug-in and **Select Channels** receives the waveform data. **Select Channels** is a plug-in parameter that references waveforms from other FlexLogger channels the user specified in the **Plug-in Configuration** dialog box. Once the data is read, the **Get Data from Stream** VI parses specific waveforms based on the unique names used when creating the channels.

Read Digital Setpoint VI gets all of the digital setpoint and waveform data and returns a single point of the most recent data for all channels.

Below is the resulting channel table in FlexLogger.

New FlexLogger IO Plug-in 1 (cDAQ1Mod1/ai0, cDAQ1Mod1/ai1, cDAQ1Mod1/ai2, cDAQ1Mod1/ai3, cDAQ1Mod1/ai4)				
	CHANNEL NAME	LIVE VALUE	DETAILS	ALARMS
Scaled	Scaled Max Waveform	109.049		
▼ Settings				
	CHANNEL NAME	LIVE VALUE	DETAILS	
Enable	Enable	High		
Mult	Multiplier	10.0000		
Offset	Offset	200.000		

Alternative Configuration - Plug-in Parameters

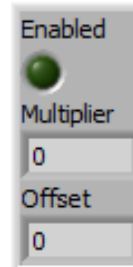
About this task

If you do not need to change settings while logging data, you can make Enabled, Multiplier, and Offset parameters of the plug-in instead of setpoint channels. Reconfigure the code from the previous example as follows:

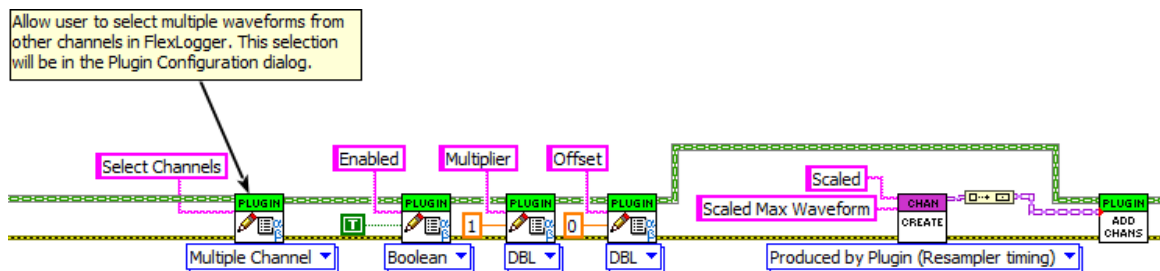
Procedure

1. Update the class control to contain private data that can be passed between the VIs.

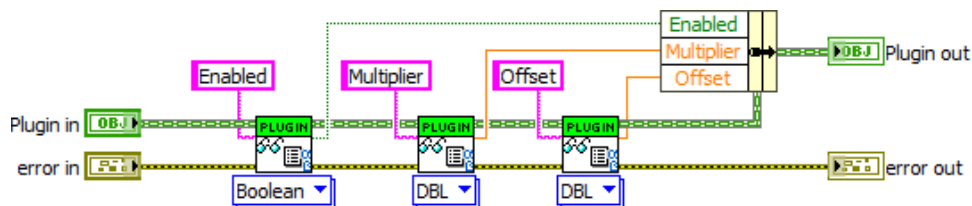
Cluster of class private data



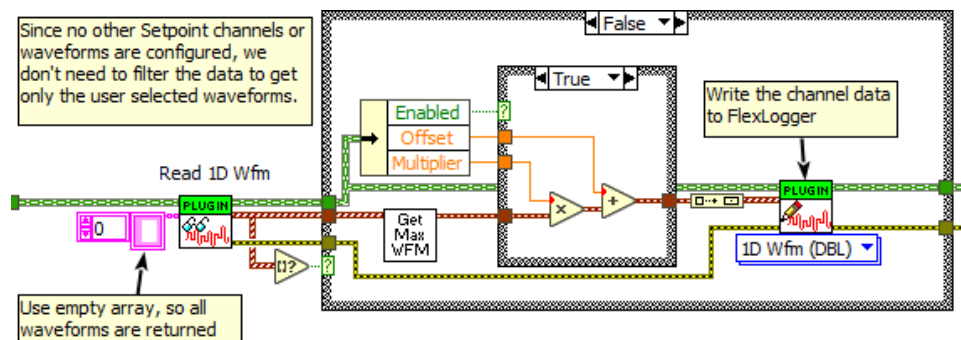
2. Modify Initialize VI to create the parameters for the plug-in.



3. Read the parameter values in Configure Session VI and update the private data. This VI is called whenever the user changes the parameter values from the **Plug-in Configuration** dialog box in FlexLogger.



4. Modify Process VI to use the Enabled, Multiplier, and Offset private data instead of parsing it out of the waveform data.



Below is the resulting plug-in configuration and channel table in FlexLogger:

Configure: New FlexLogger IO F

Name

New FlexLogger IO Plugin 1

Select Channels

 Multiple sources mapped

☒ Enabled

Multiplier

10

Offset

200

OK

Cancel

 New FlexLogger IO Plugin 1 (cDAQ1Mod1/ai0, cDAQ1Mod1/ai1, cDAQ1Mod1/ai2)  			
	CHANNEL NAME	LIVE VALUE	DETAILS
Scaled	Scaled Max Waveform	242.634	

Alternative Configuration - Dynamic Channel Number with Individual Settings

About this task

You can configure the settings in FlexLogger to support a dynamic number of channels with separate settings. In this configuration, each channel has its own offset and multiplier setting as well as user-selected waveforms that each channel applies unique scaling parameters to. Reconfigure the code from the first example as follows:



NOTE
You can disable an entire channel in FlexLogger instead of keeping a separate channel parameter to disable it. To disable a channel, select the channel and click the **Disable** button on the right of the channel table.

 New FlexLogger IO Plugin 1  					
	CHANNEL NAME	LIVE VALUE	DETAILS	ALARMS	
Scaled 1	Scaled Max Waveform 1	32.8087	Multiplier: 3, Offset: 22, Select Channels: cDAQ1Mod1/ai2		 
Scaled 2	Scaled Max Waveform 2	Disabled	Multiplier: 1, Offset: 0, Select Channels: cDAQ1Mod1/ai7		Disable
Scaled 3	Scaled Max Waveform 3	265.078	Multiplier: 1, Offset: 0, Select Channels: cDAQ1Mod1/ai6, cDAQ1Mod1/ai5		

Procedure

1. Update the class private data control to track the number of channels.

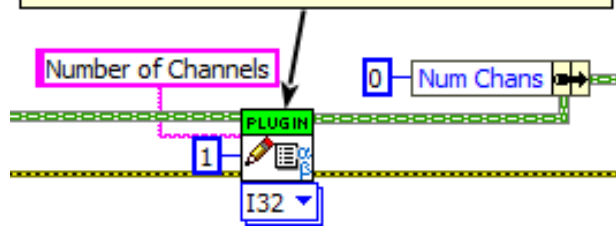
Cluster of class private data

Num Chans

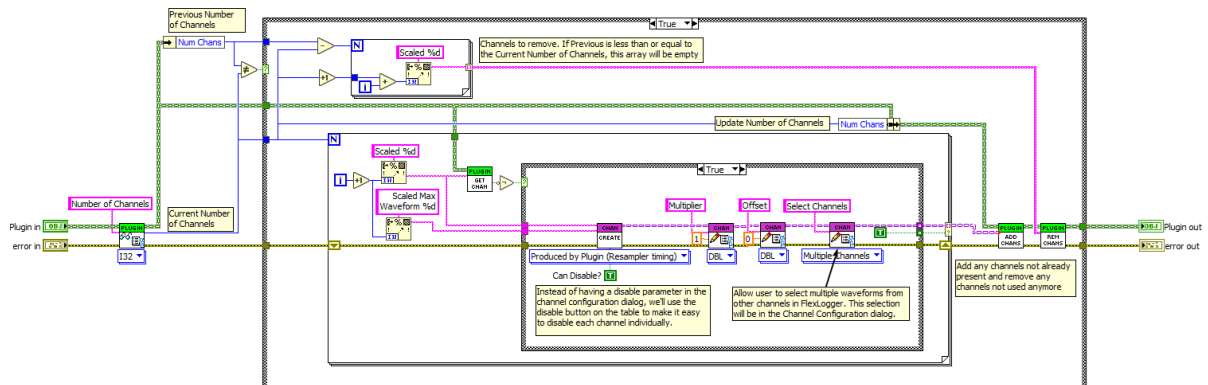
0

2. Modify Initialize VI to create the Number of Channels parameter for the plug-in and initialize the private data.

Only configure Number of Channels on the Plug-In. The channels will be created in the Configure VI since that is called everytime a parameter changes and this VI is only called once.



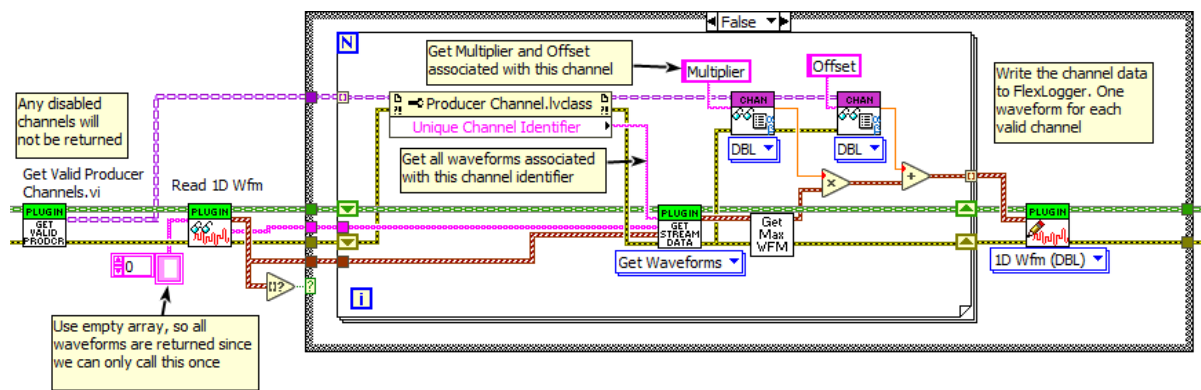
3. In the Configure Session VI, read the current number of channels configured by the user and compare with the previous value stored in the private data. Add any new channels that need to be configured and remove any channels that are no longer needed.



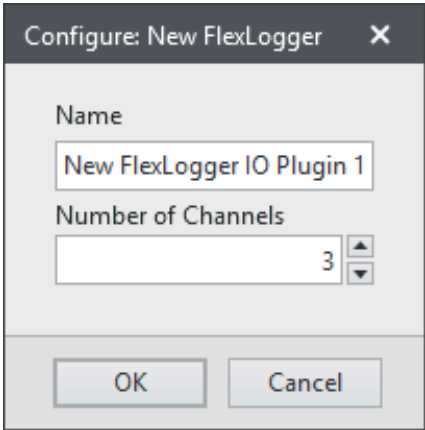
The **Get Channel With Name VI** (<vi.lib>\FlexLogger\SDK\PluginSDK.lvlib\Plugin\Get Channel With Name VI) determines if the channel has already been created. Since you allow the user to disable channels, disabled channels are not returned by the **Get Valid Channels VI**. You determine which channels to add and remove to prevent any existing channels from being removed or overwritten. This ensures all the configured settings for any existing channels remain unchanged, including the disabled setting. **Get Valid Channels VI** is called whenever the user changes the Number of Channels parameter from the **Plug-in Configuration** dialog in FlexLogger.

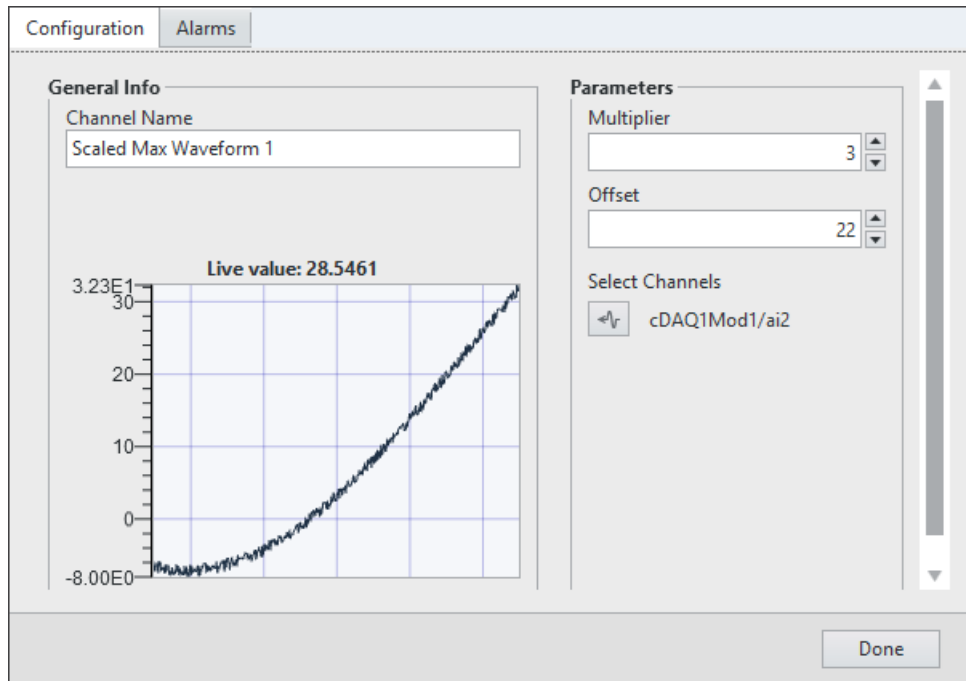
4. Modify the **Process VI** to go through all enabled channels. Use **Get Valid Producer Channels VI** (<vi.lib>\FlexLogger\SDK\PluginSDK.lvlib\Plugin\Get Valid Producer Channels VI) to obtain only enabled channel producers for

this plug-in. For each channel, get the waveforms, multiplier, and offset associated with the channel.



Below is the resulting plug-in Configuration, Channel Configuration and channel table in FlexLogger:





	CHANNEL NAME	LIVE VALUE	DETAILS	ALARMS	
Scaled 1	Scaled Max Waveform 1	45.7509	Multiplier: 3, Offset: 22, Select Channels: cDAQ1Mod1/ai1,cDAQ1Mod1/ai2		
Scaled 2	Scaled Max Waveform 2	Disabled	Multiplier: 1, Offset: 0, Select Channels: cDAQ1Mod1/ai0		
Scaled 3	Scaled Max Waveform 3	8.89367	Multiplier: 1, Offset: 0, Select Channels: cDAQ1Mod2/ai0,cDAQ1Mod2/ai1		

Create Channel VI

The Create Channel VI allows you to create three types of channels, **Produced by Plug-in**, **Produced by Plugin (Resampler timing)**, and **Setpoint consumed by Plugin**. Creating channels involves the following information:

Table 8 : Plug-in Channel Components

Name	LabVIEW Data Type	Description
Unique Channel Identifier	string	Channel identifier that is unique in the plug-in context. Displayed in the far-left Channel Specification column (for example, ai0 for a DAQ channel). Strings with five or fewer characters display best in FlexLogger.
Default Channel Name	string	Channel name in FlexLogger.
Display Group	string	Name used in Channel Specification header and channel selector.
Unit	string	Channel units (for example, Hz).

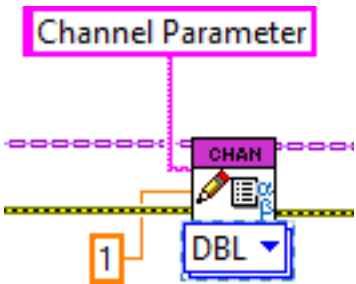
Table 8 : Plug-in Channel Components (Continued)

Name	LabVIEW Data Type	Description
Can Delete?	Boolean	Allow users to disable this channel. False by default. When setting Can Disable? to True, plug-in developers should monitor which channels are disabled by using Get Valid Channels . When a channel is disabled, do not write data back to FlexLogger for the disabled channel.
Default Value	double	Setpoint consumed by Plugin channels allow plug-in developers to set the default value that appears in FlexLogger when the plug-in is added.
dt(sec)	double	If Produced by Plugin channels use the 1D Wfm (DBL) or 1D Wfm (Digital) option provided by Write VI , this parameter tells the FlexLogger application what dt to expect.

FlexLogger allows users to change a channel’s **Default Channel Name** from both the channel configuration dialog and directly in the Channel Specification table. Additionally, FlexLogger requires all channels in the Channel Specification to have unique names—and will automatically change a plug-in channel name if that rule is violated (for example, myChannel becomes myChannel_1). Regardless of the change to **Default Channel Name**, the **Unique Channel Identifier** parameter remains unchanged.

Channel Specific Parameters

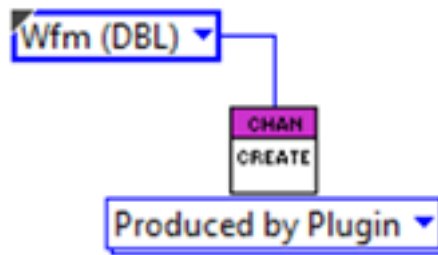
The Channel class supports double-precision floating point numbers, strings, enums, Booleans, and integers as channel-level parameters. Channel-specific parameters can be created by using the Channel class’s **Write Parameter VI**, as shown in the following example:



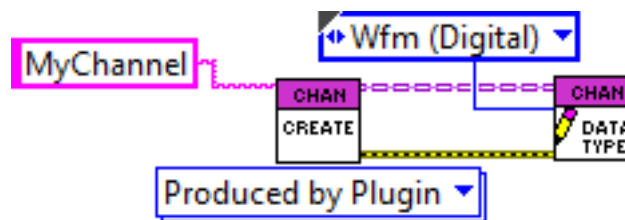
Channel Data Types

The Channel class supports analog (Wfm (DBL)), digital (Wfm (Digital)), and string data types for all channels. Analog waveform is the default data type.

To specify a data type, connect it to the **Create Channel VI**.



You can also set the data type using **Write Data Type VI** and connecting it to the **Create Channel VI**. The following example shows how to use **Write Data Type VI** to change the data type of a channel.

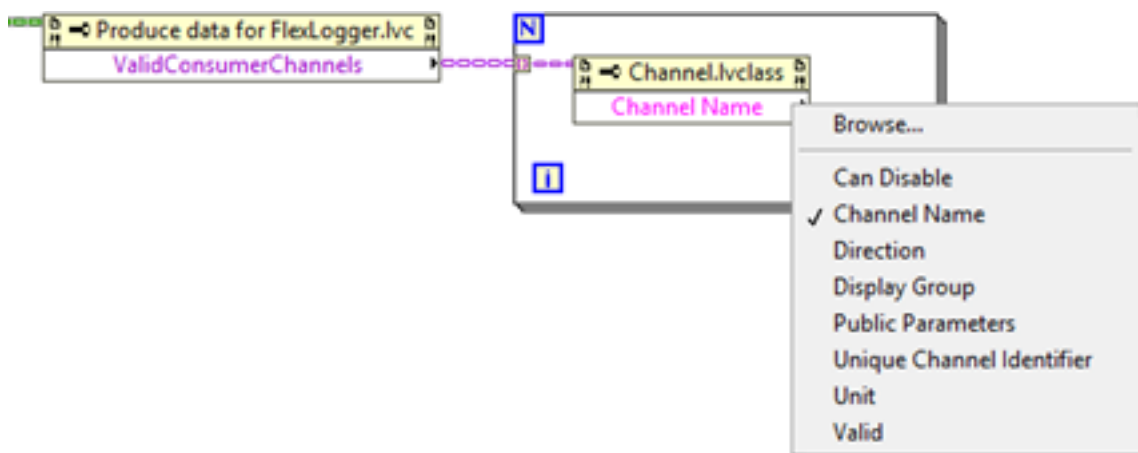


NOTE

FlexLogger cannot send string data to a plug-in.

Dynamic Channels

To help developers work with dynamic channels—such as when names change or channels are disabled—plug-ins maintain an array of valid channels. Wiring a property node to a plug-in class wire provides direct access to the `ValidProducerChannels`, and `ValidConsumerChannels` arrays:



Knowing what channels are valid allows developers to configure their plug-ins correctly and produce the right data for the available channels.

When channels assigned to a plug-in need to be changed, developers can use **Replace Producer Channels VI** and **Replace Channel Group VI** to replace multiple channels at once rather than individually. **Replace Producer Channels VI** replaces all

existing producer channels with a specified list of channels, which can be useful in situation where the user configuration changes in a way that requires a completely new set of producer channels. Replace Channel Group VI replaces all channels in a particular group with a specified list of channels, which can be useful in situations where only a subset of the channels need to be replaced.

System Interface Class

The System Interface class (`System Interface.lvclass`) defines the execution environment interacting with a FlexLogger plug-in. When FlexLogger loads and runs a built plug-in, the plug-in will communicate with FlexLogger’s underlying data engine. When debugging a plug-in in the Plug-in Environment Simulator, the plug-in will communicate with a simplified, test-specific environment.

Additional Classes

The following classes belong to `PluginSDK.lvlib` and are outside the scope of FlexLogger plug-in development. These classes should not be used in your plug-ins.

Table 9 : Additional Classes Not Used in Plug-in Development

Class Name	Summary
<code>Addon Interface.lvclass</code>	Defines the communication between plug-ins and FlexLogger
<code>Message Completion Handler.lvclass</code>	Messaging functionality for classes that do not inherit from <code>Plugin.lvclass</code>
<code>Parameters.lvclass</code>	Provides the parameter functionality for the Channel and Plug-in classes

FlexLogger Plug-In Debugging

Troubleshoot plug-in problems with the debugging resources available in LabVIEW and FlexLogger.

Refer to the following debugging resources that align with your troubleshooting scenario.

Table 10 : Plug-In Debugging Resources

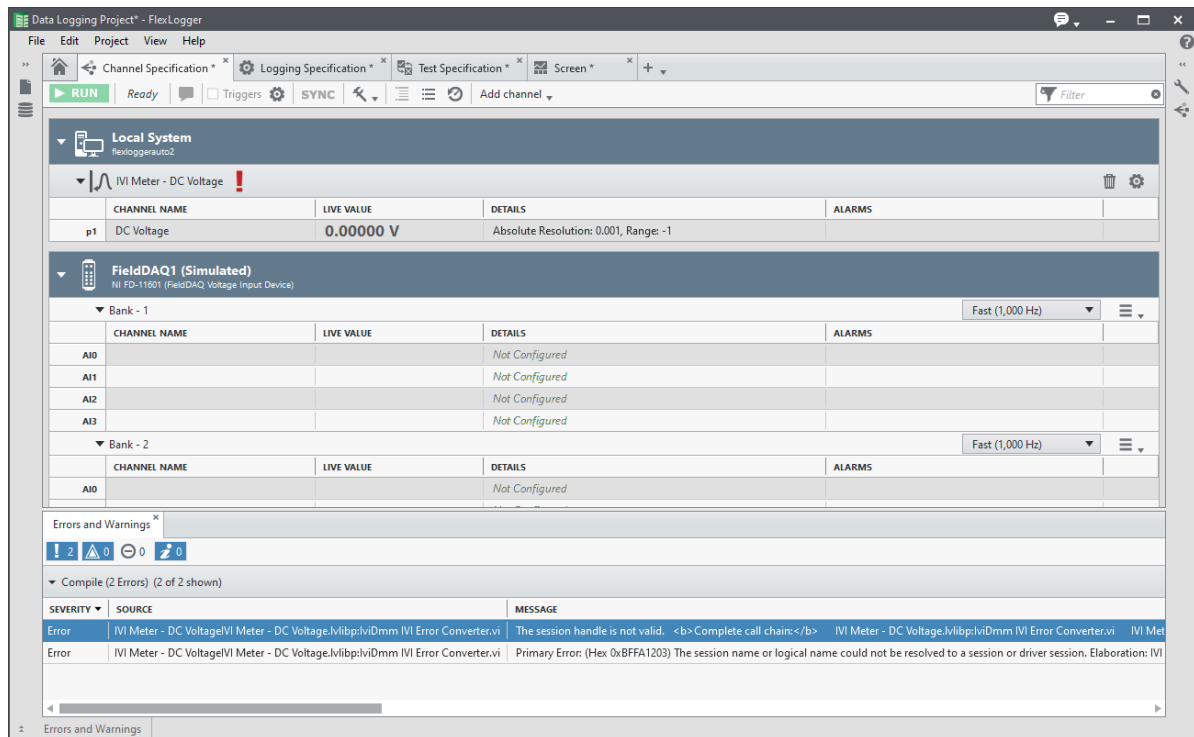
Scenario	Resource
Why is my plug-in is not available to load in FlexLogger?	Review the error log created by FlexLogger. Refer to <i>Debugging Plug-In Loading Errors</i> for more information.
How do I view errors published by VIs used in my plug-in in FlexLogger?	Configure your VIs in LabVIEW, and view errors and warnings in FlexLogger. Refer to <i>Viewing Errors in FlexLogger</i> for more information.
How do I interactively test my plug-in before I use it in FlexLogger?	Use the Plug-in Environment Simulator in LabVIEW. Refer to <i>Interactively Testing Plug-Ins</i> for more information.

Debugging Plug-In Loading Errors

If FlexLogger fails to parse a plug-in XML file, the plug-in is not available for you to add in the Channel Specification. If a parsing failure occurs, FlexLogger creates a `Error.txt` file in the plug-in folder. The file contains details about the problem.

Viewing Errors in FlexLogger

Because the plug-in class provides built-in error handling, it is important that developers wire their error wires to the provided error out terminals. For example, IVI class driver VIs publish errors on their error wires when the VIs are initialized incorrectly. Wiring the error terminals from the driver VIs to the error out terminal ensures errors appear in FlexLogger as shown in the following figure.

Figure 1 : FlexLogger Error Messages

In FlexLogger, three distinct error notifications appear.

- Detailed errors in the **Errors and Warnings** pane
- An error icon on the plug-in header
- The Error indicator next to the **Stop Test** button

Interactively Testing Plug-Ins

When plug-ins load and run in FlexLogger, the application's compiled data engine and the compiled plug-ins prevent interactive debugging with LabVIEW. The Plug-in Environment Simulator provides a simplified, test-specific environment that allows developers to run and debug plug-ins from the source G code.

The Plug-in Environment Simulator installs to %ProgramFiles%\National Instruments\LabVIEW <version>\resource\FlexLogger\SDK\Plugin Environment Simulator.

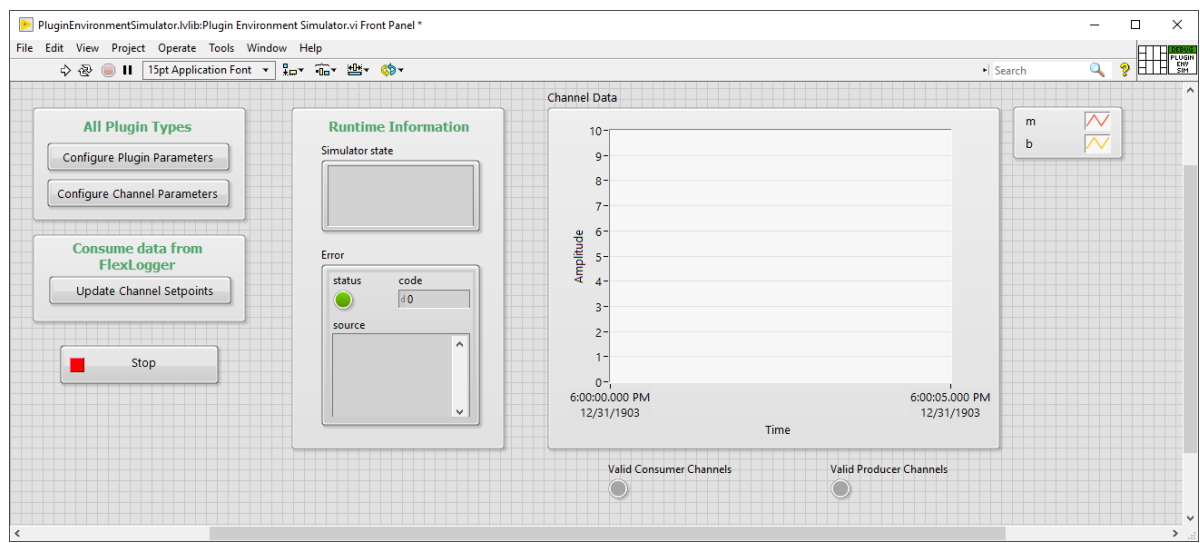


NOTE

The Plug-in Environment Simulator does not support digital channels.

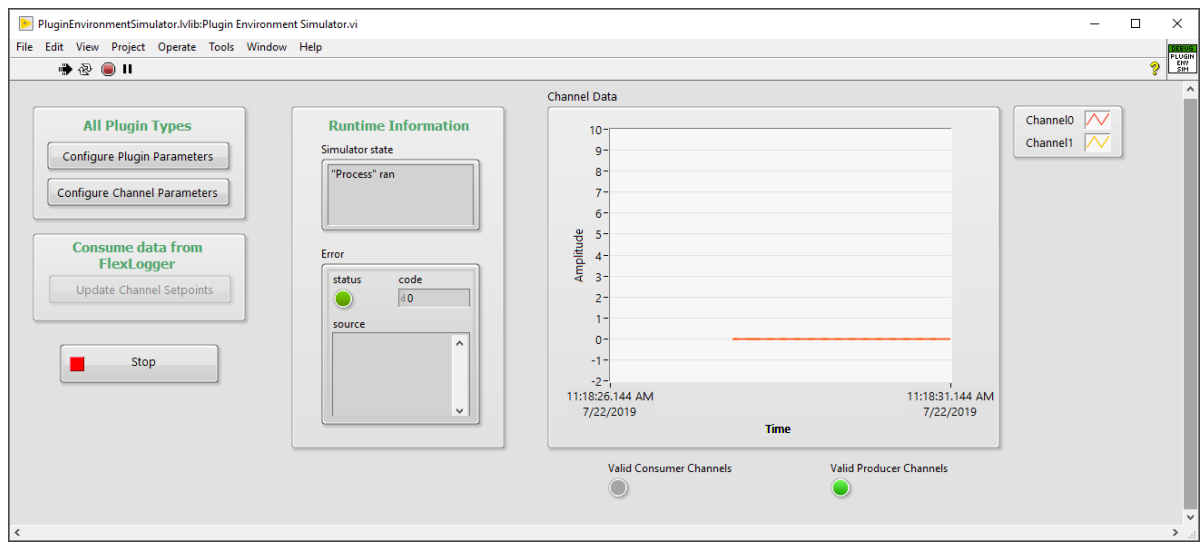
To enable interactive debugging of plug-in source code, the Simulator is provided as a VI as shown in the following figure.

Figure 2 : Plugin Environment Simulator VI

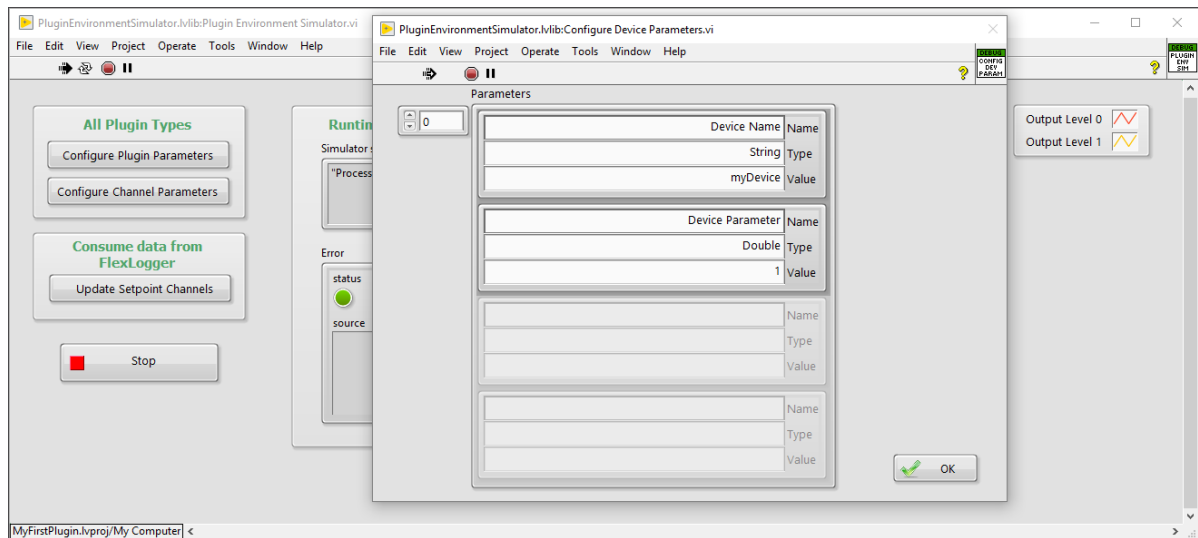


To debug a plug-in, open and run the Plugin Environment Simulator VI. When the simulator starts running, it prompts you for the plug-in class to debug. The following figure displays the simulator running an unmodified Produce Data for FlexLogger plug-in.

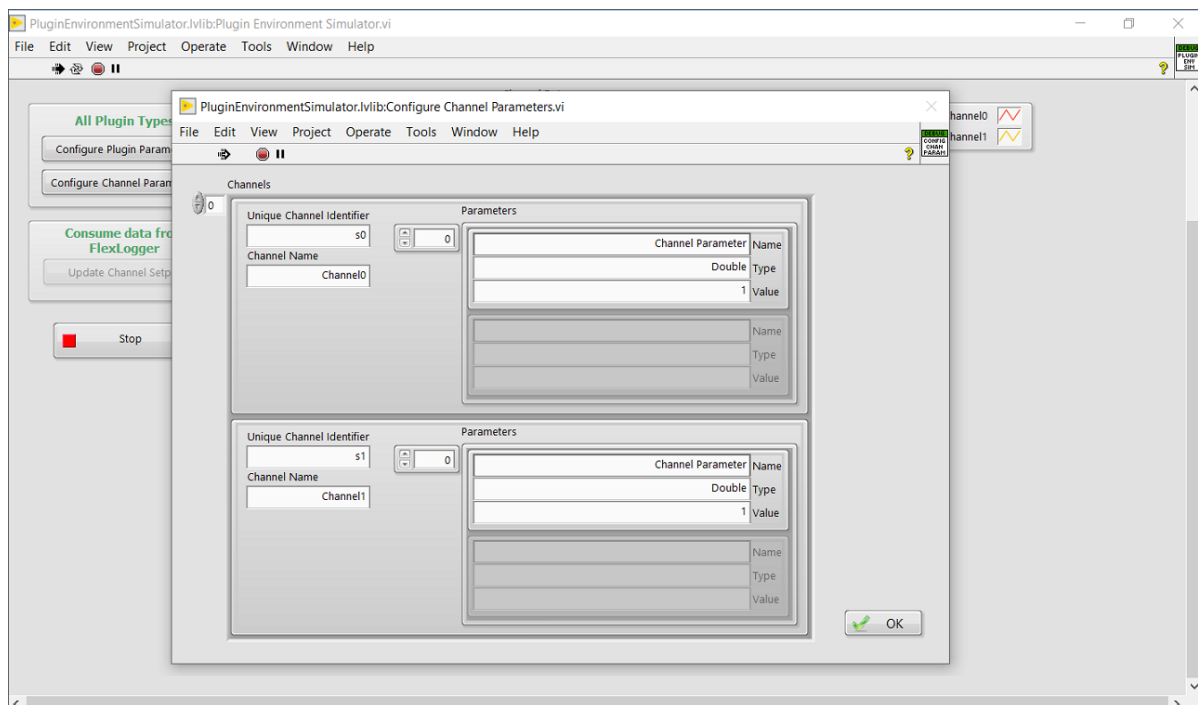
Figure 3 : Plugin Environment Simulator — Produce Data for FlexLogger Plug-In



The **Configure Plugin Parameters** button allows you to update plug-in parameters.

Figure 4 : Configure Plugin Parameters

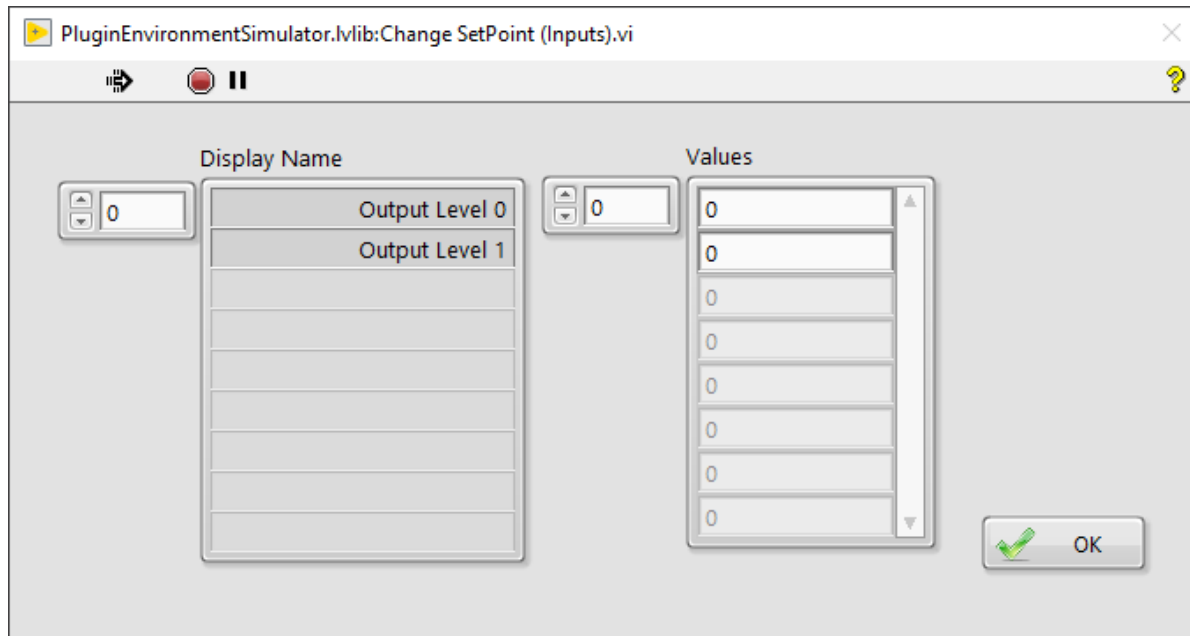
Similarly, the **Configure Channel Parameters** allows developers to update channel parameters.

Figure 5 : Configure Channel Parameters

Plug-ins following the consume setpoint data template do not write any data to FlexLogger, so the **Channel Data** chart shows the current setpoint values. The **Update Setpoint Channels** button provides the input information needed to write the correct

values to an external system. The following figure shows setting the setpoint for an unmodified setpoint consumer plug-in example.

Figure 6 : Update Setpoint Channels



When using waveform channel inputs with the simulator, it is first necessary to pick channel(s) that are mapped to the parameters. This mimics the behavior when interacting with the application. In the simulator, any designated name receives a constant value waveform. Configure mapped channels through the **Configure Device Parameters** dialog as shown in the following figures.

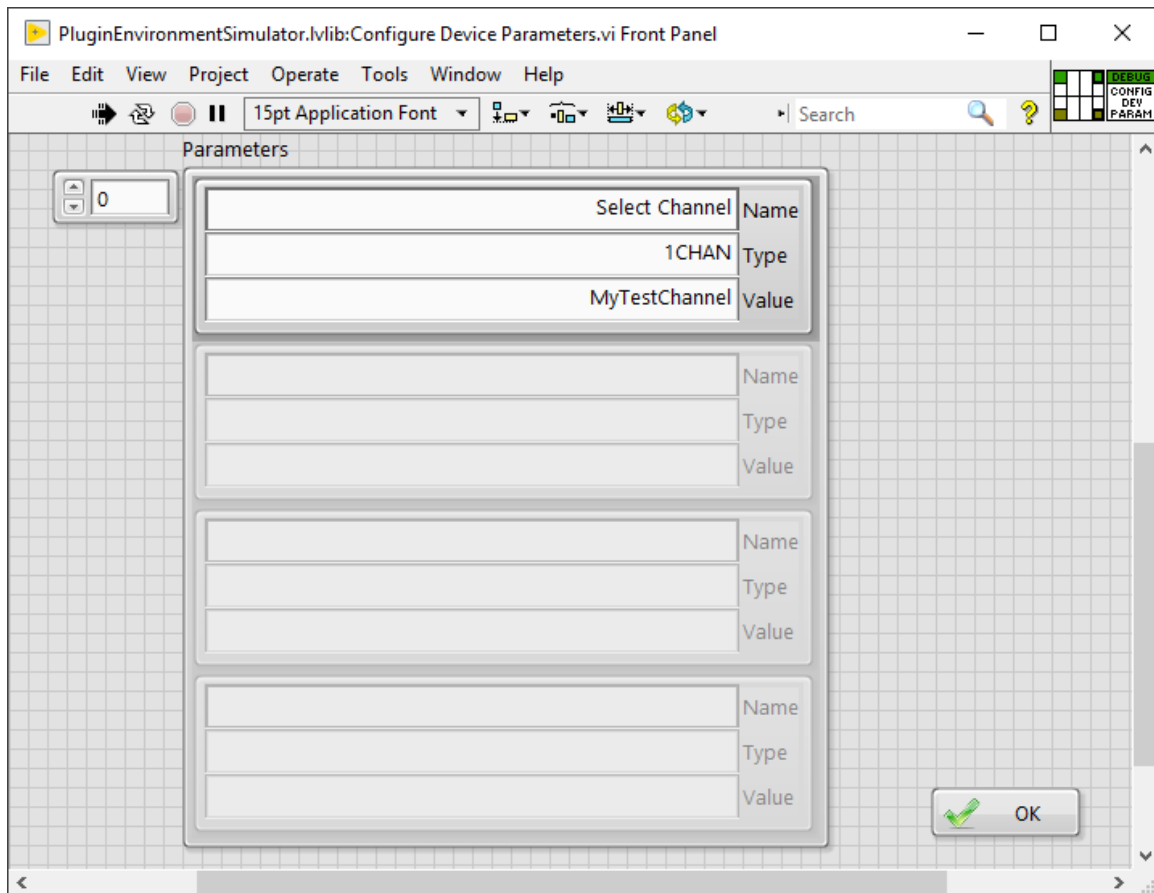
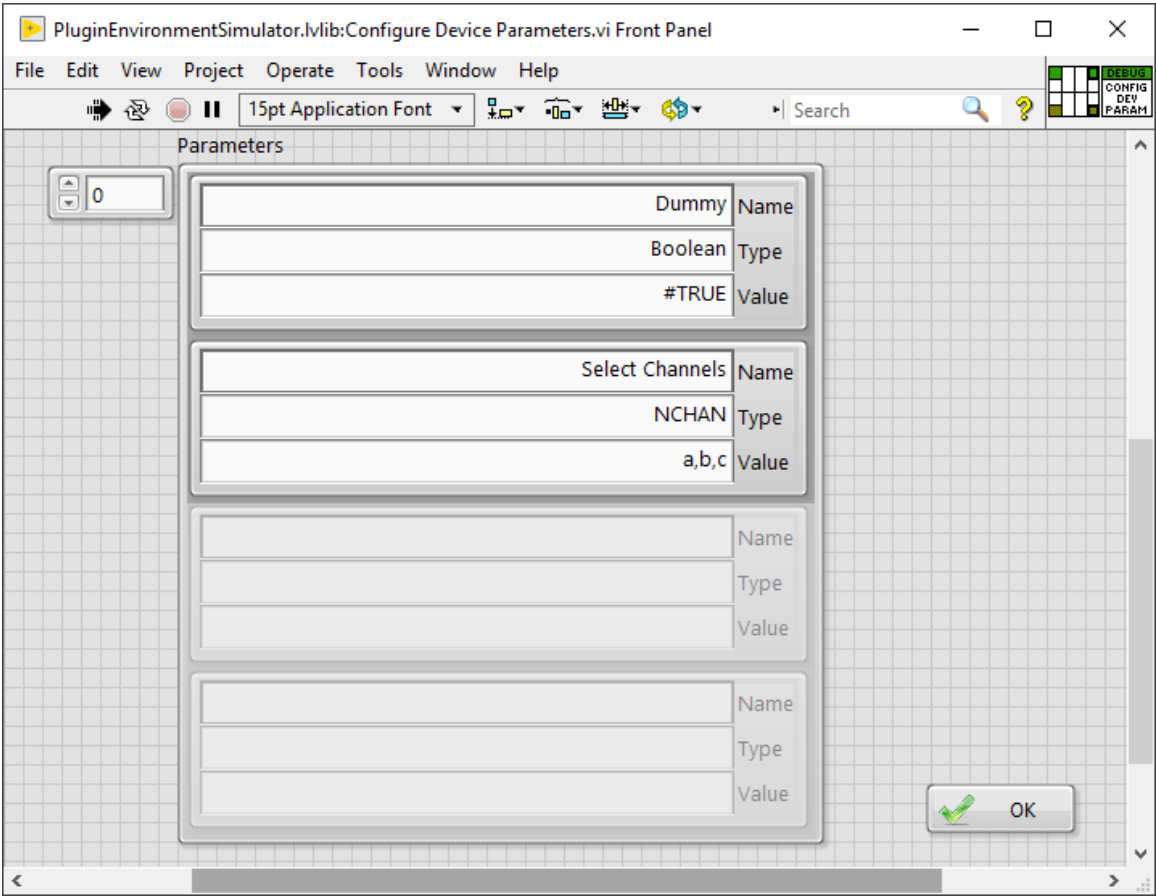
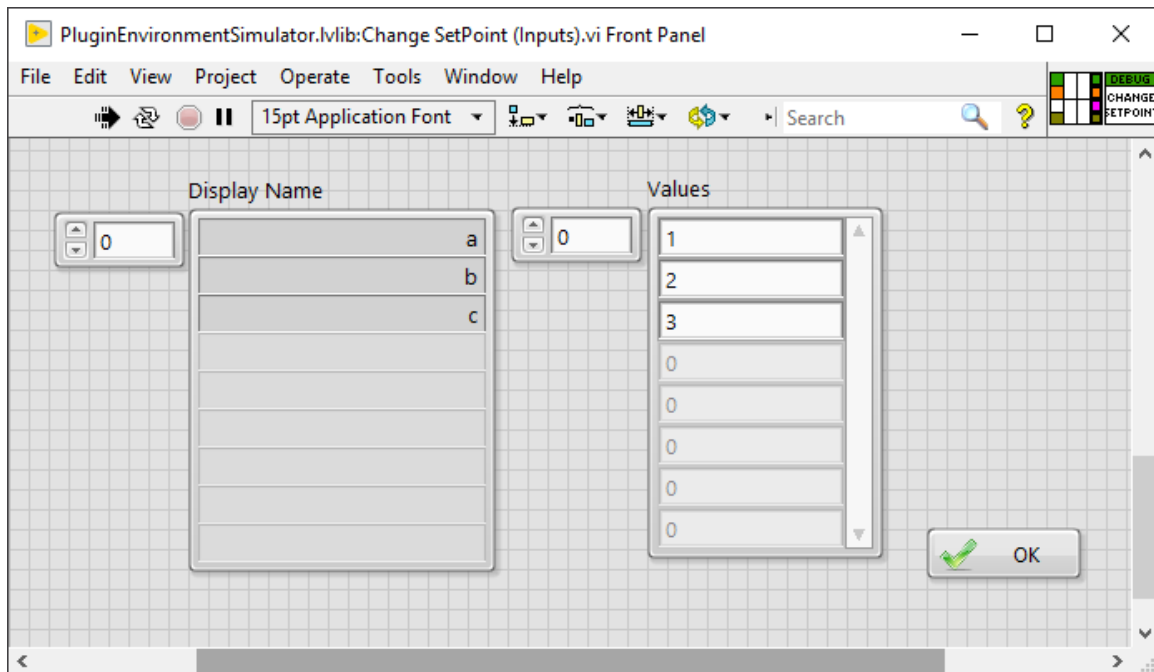
Figure 7 : Configure Device Parameters

Figure 8 : Configure Device Parameters



You can change the constant value using the **Change Setpoint** option.

Figure 9 : Change Setpoint Option

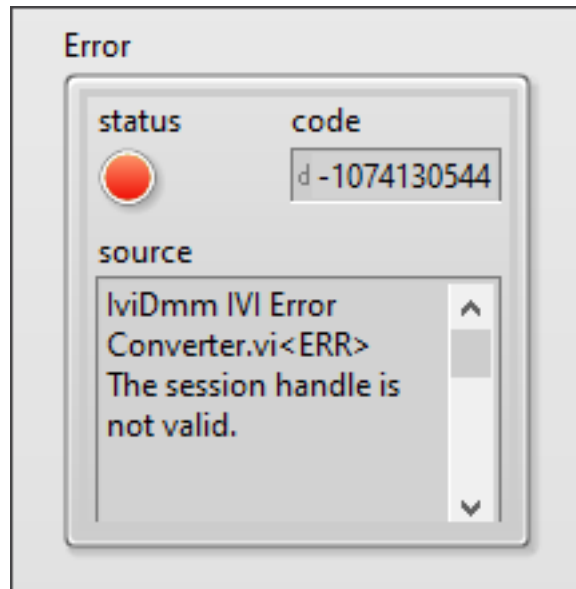
Interactive Debugging Session Example

About this task

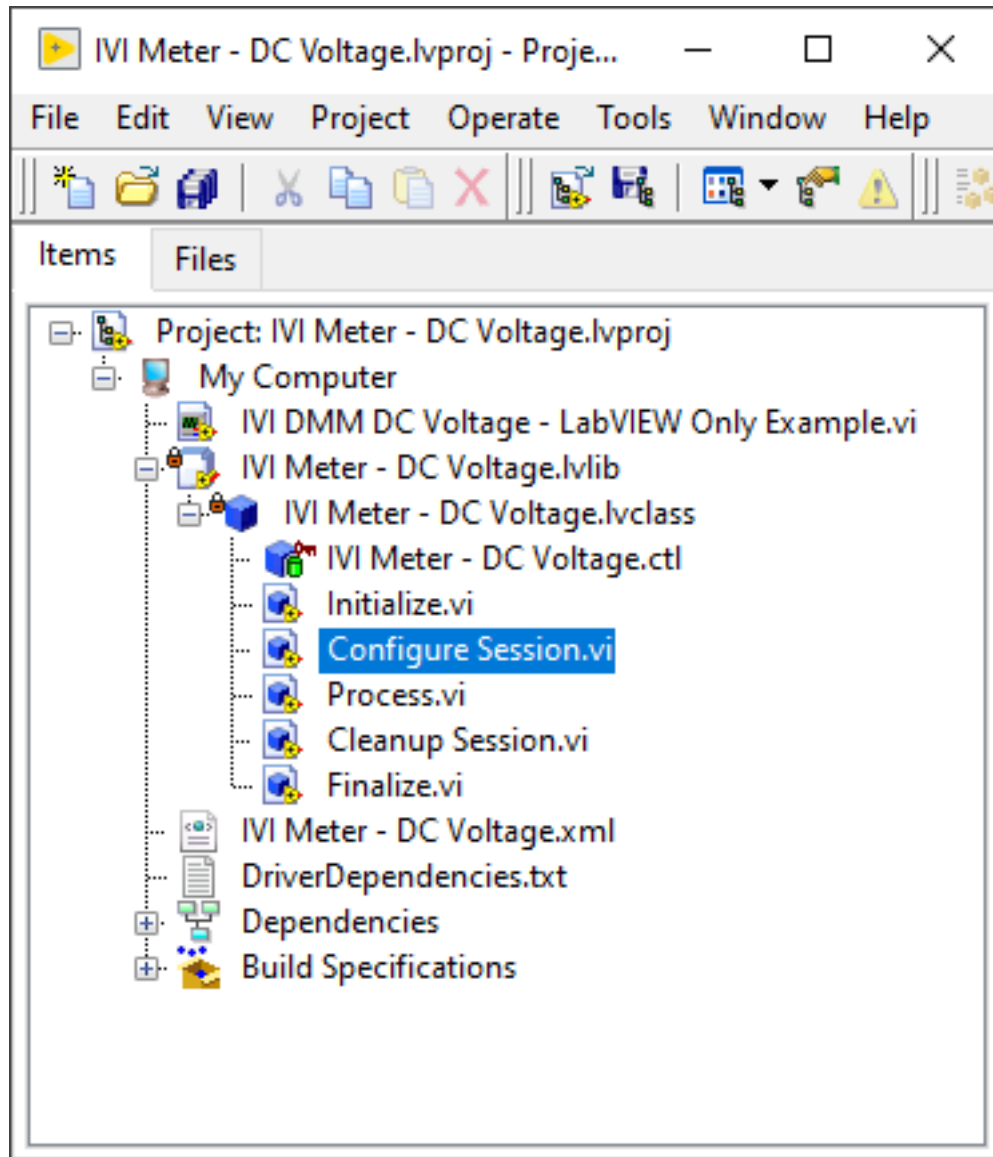
The following scenario illustrates how the Simulator could be used to investigate an issue with a plug-in under development.

Procedure

1. After loading and configuring the DMM plug-in, the plug-in channel does not produce any data. The **Error and Warning** pane in FlexLogger, displays an error message that references a problem in the `Configure Session VI`.
2. To investigate further, open the LabVIEW project for the DMM plug-in
`%ProgramFiles%\National Instruments\LabVIEW <version>\examples\FlexLogger\IO Plugins\IVI Meter - DC Voltage\IVI Meter - DC Voltage.lvproj`.
3. Open and run `Plug-in Environment Simulator VI` and then select the LabVIEW class of the DMM plug-in (`IVI Meter - DC Voltage.lvclass`).
4. After updating the plug-in parameter that defines the **IVI Logical Name** to test channel, the simulator stops running and the **Runtime Information** error terminal displays an error:



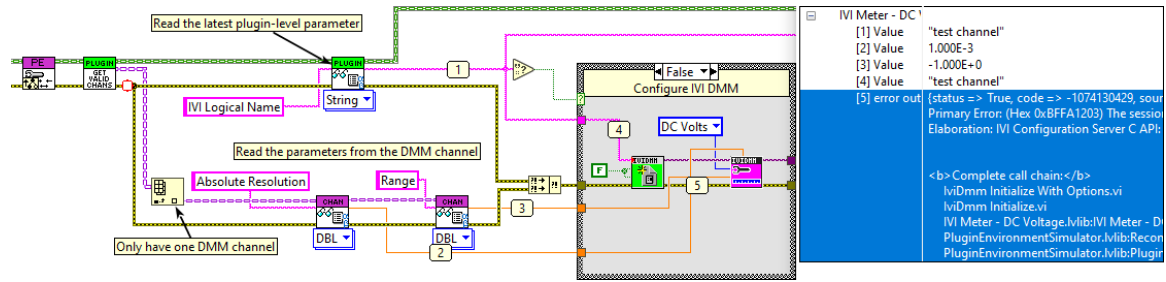
5. To investigate further, navigate to the plug-in LabVIEW project and open Configure Session VI.



6. With **Configure Session VI** open, add a breakpoint to see what part of the logic is generating the error:



7. With the breakpoint in place, run the simulator again and load the DMM plug-in. That action will hit the newly placed breakpoint. Using LabVIEW's debugging tools, step through the block diagram to see exactly which VI is having problems:



8. After this debugging session, you will see that the plug-in has been using the wrong **IVI Logical Name** parameter, resulting in an error from the IVI driver VIs. The plug-in runs as expected when using the correct name.

Information is subject to change without notice. Refer to the NI Trademarks and Logo Guidelines at ni.com/trademarks for information on NI trademarks. Other product and company names mentioned herein are trademarks or trade names of their respective companies. For patents covering NI products/technology, refer to the appropriate location: Help»Patents in your software, the patents.txt file on your media, or the National Instruments Patent Notice at ni.com/patents. You can find information about end-user license agreements (EULAs) and third-party legal notices in the readme file for your NI product. Refer to the Export Compliance Information at ni.com/legal/export-compliance for the NI global trade compliance policy and how to obtain relevant HTS codes, ECCNs, and other import/export data. NI MAKES NO EXPRESS OR IMPLIED WARRANTIES AS TO THE ACCURACY OF THE INFORMATION CONTAINED HEREIN AND SHALL NOT BE LIABLE FOR ANY ERRORS. U.S. Government Customers: The data contained in this manual was developed at private expense and is subject to the applicable limited rights and restricted data rights as set forth in FAR 52.227-14, DFAR 252.227-7014, and DFAR 252.227-7015.